cubepals

TECHNICAL ARCHITECTURE

Blockly architecture

How Blockly runs Minecraft servers

Document	Technical architecture
Revision	7
Date	19 October 2026
Status	Current
Rendered from	`09bff66` on `claude/public-docs-licensing-yx622s`

Document control

How Blockly runs Minecraft servers: the control plane, its runtimes, the fleet and its node daemon, storage, the edge, money, and how all of it is operated. This is the canonical technical description. Where it and the code disagree, the code is right and this document has a bug.

Title	Blockly architecture
Document type	Technical architecture
Revision	7, 12 October 2026
Status	Current
Source	`09bff66` on `claude/public-docs-licensing-yx622s`, rendered 2026-10-19
Scope	The code on `main`: the control plane, the web app and the edge; the Fly, Boat, fleet and Docker runtimes; blocklyd; several runtimes in one deployment, the fleet canary and running admission.
Readers	A maintainer returning to the code, a new engineer, someone judging the infrastructure, an operator on call, and anyone asking why it is built this way.

Revisions

Revision	Date	Commits	Changes
1	2026-10-01	`4b73704 to 3da68bb`	First complete version: every part, diagram and appendix.
2	2026-10-02	`d6a6e9f to 22c066b`	What the fixes before merge changed; the test counts on the final code.
3	2026-10-02	`09556cf to 4ebf721`	Archives larger than one PUT go to the store in parts.
4	2026-10-02	`f44172b`	Branded and formalized: a cover, document control, how to read it, and a running header and footer. What the document says is unchanged.
5	2026-10-08	this revision: `09bff66`	Everything described is on `main`, so the branch labels are gone. Choosing a runtime is explained by its technical reasons, and every citation points into the repository.
6	2026-10-09	this revision: `09bff66`	blocklyd moved to its own repository, `cubepals/blocklyd`, and the control plane pins a release of it. Its citations start `blocklyd/`.
7	2026-10-12	this revision: `09bff66`	Business matters are out: prices, vendor trials and the roadmap. The mechanisms stay.

Contents

How to read this document . . . 4

PART I · ORIENTATION

1 Executive overview . . . 5
2 System map . . . 5
3 Repository and module map . . . 6

PART II · RUNTIMES

4 The MinecraftRuntime abstraction . . . 8
5 Several runtimes in one deployment . . . 9
6 FleetRuntime . . . 9
7 blocklyd . . . 10
8 One machine, many servers . . . 11
9 Placement . . . 12

PART III · FLEET STATE AND SAFETY

10 State ownership . . . 13
11 Desired state and reconciliation . . . 13
12 Enrollment . . . 14
13 Heartbeats and node health . . . 15
14 Epochs and fencing . . . 15
15 Execution leases . . . 16
16 Workload lifecycle . . . 17
17 The startup path . . . 17
18 Sleep and wake on Boat, Fly and the fleet . . . 18

PART IV · DATA

19 The storage model . . . 19
20 The backup pipeline . . . 20
21 The restore pipeline . . . 21
22 Relocation . . . 21
23 Draining . . . 22
24 Node loss . . . 23
25 Split brain . . . 24

PART V · NETWORK AND PROVIDERS

26 Edge routing . . . 25
27 Provider layering . . . 26
28 Boat, as it actually is . . . 27
29 Direct-host providers . . . 27
30 Choosing a runtime . . . 27
31 The canary . . . 27

PART VI · SECURITY AND OPERATIONS

32 Security model . . . 29
33 Observability . . . 29
34 Operator workflows . . . 30
35 Deployment topology and trust boundaries . . . 30
36 Failure matrix . . . 31

PART VII · STATUS

37 Status . . . 33
38 Current limitations . . . 33

APPENDICES

A Interfaces . . . 35
B Database entities . . . 36
C Protocol operations . . . 37
D Configuration and environment . . . 38
E Metrics and events . . . 39
F Benchmarks and measurements . . . 40
G Test coverage . . . 40
H Glossary . . . 41

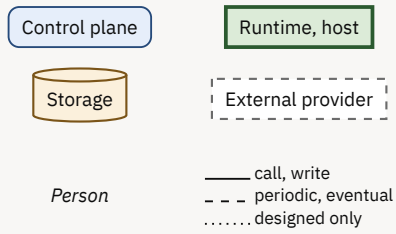
How to read this document

Every capability this document describes carries a status label where it is described: how far it exists, not whether it is deployed. Sections are numbered and appendices lettered; a reference such as §12 links to its section, and figures and tables in the body are numbered and captioned. Claims a reader might doubt cite the code as `path:line`, at the commit on the cover.

Status labels. Every capability carries one.

Label	Meaning
IMPLEMENTED	In the code, on the product's own paths. Whether it is deployed is said separately.
CANARY ONLY	In the code, reached only by servers an operator sends to the fleet.
DESIGNED	Written down, not built.
DEFERRED	Left out on purpose, for now; the reason is given.
RESEARCH ONLY	Measured or studied; nothing in the product depends on it.
HISTORICAL	True once; kept so old documents make sense.

Drawing conventions. They hold in grayscale.



Where to start

If you are	Read first
A maintainer returning to the code	§1, §37, §38; then any part you are changing.
A new engineer	§1 to §4, §16 and §17, then the part you will work on, with §A at hand.
Judging the infrastructure	§1, §6, §8, §30, §32, §37, §38.
An operator on call	§13, §24, §25, §34, §36, and §D.
Asking why	The "Why" boxes: every major decision has one, next to what it explains.

PART I

Orientation

What Blockly is, the map of the whole system, and where its code lives.

1 Executive overview

Blockly is Minecraft server hosting that feels simple. A player names a server, picks what to play, invites friends, and plays; everything else is Blockly's to decide. That rule shapes the architecture: Blockly absorbs complexity so the player doesn't have to, and asks only about what the player alone knows, what is destructive or one-way, money, and who can reach the server (`AGENTS.md`).

The system in seven sentences.

1. A **control plane** (TypeScript, one image in three roles: `api`, `worker`, `realtime`) owns every server's desired state in Postgres and is the only writer of it.
2. Owners act through a **web app** (Next.js) that calls the control plane's tRPC API; every change that touches infrastructure becomes an **operation** on a per-server FIFO queue, run by workers.
3. Workers reach compute only through **one port**, `MinecraftRuntime`. Behind it sit adapters for Fly Machines, Boat sandboxes, Blockly's own **fleet** of machines, local Docker, and a fake for tests.
4. Players connect to one **edge** (mc-router with a small agent) that routes each server's hostname to wherever it runs, answers for sleeping servers, and wakes them on join.
5. Worlds live on the runtime's disk while a server runs, with **durable copies** in an S3-compatible archive store; idle worlds rest there with no compute at all.
6. The fleet runtime drives **blocklyd**, a small Rust daemon on each machine, over mutual TLS, with placement epochs, fences and an execution lease instead of a cluster.
7. Money is plans with included hours, billed through Polar; extra paid hours are deferred.

Table 1: Where things stand at the commit this document was rendered from. The full table is §37.

Part	Status	In one line
Control plane, web, edge, Fly runtime	IMPLEMENTED	What staging runs (made by `scripts/staging.ts` on Fly) and what the production Terraform configures (`RUNTIME_PROVIDER=fly`).
Boat runtime	IMPLEMENTED	Configured in no environment (§28).
Several runtimes at once, the canary	IMPLEMENTED	Opt-in: nothing is configured to use it.
Fleet runtime and blocklyd	IMPLEMENTED CANARY ONLY	Tested end to end on one VM; never run on independent machines.
Running admission	IMPLEMENTED	Sleeping fleet servers hold no node memory.
Automatic failover, provider fencing, cost-aware placement	DEFERRED	Deliberately not built; the reasons are in §25 and §9.
ZFS snapshots	RESEARCH ONLY	Evaluated, not adopted (§19).

Decisions that explain the rest.

- **A server is not a machine.** The server (name, world, settings, plan) is a durable record; compute comes and goes behind a handle. Sleep, wake, moves and provider changes are all just compute changing under a server that stays (§4).
- **One writer of intent.** Only the control plane decides what should run; providers, nodes and the edge report and obey. The exceptions are bounded: Fly and fleet nodes restart a crashed server (three tries; a node only under its lease, §15), and Boat stops a parked sandbox when its time to live ends.
- **No cluster.** The fleet has no quorum, consensus store, scheduler cluster or distributed filesystem. Correctness comes from epochs (a node refuses stale orders) and from an operator's physical fence before a host is declared lost (§14).
- **Cheap idle.** The architecture optimises for servers that sleep: a sleeping server keeps disk or object storage, never memory (§8, §30).

> CAVEAT
> **What this document is not.** It is not a promise that unbuilt parts will be built; status labels say what exists. Numbers marked as measured come from test machines or Fly staging, never from production traffic (§38).

2 System map

Every running part and what talks to what. Each box is detailed later; the trust boundaries between them are in §35.

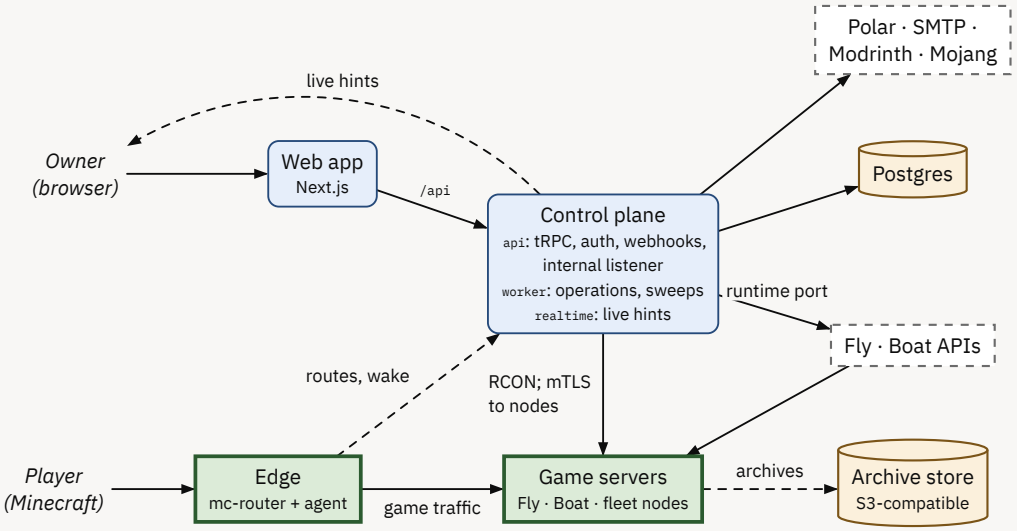


Figure 1: The system. People in italics; control plane rounded; runtime and hosts square; storage cylinders; external providers dashed.

Table 2: The parts in one table.

Part	Does	Where it runs
Web app (`apps/web`)	Owner and admin UI; proxies `/api/*` to the control plane so cookies stay host-only.	A Cloudflare Worker: `blockly-web` in production, `blockly-web-staging` on staging
Control plane (`apps/control`)	`api`: tRPC, auth, public status, webhooks, the internal listener for the edge and operators; `worker`: operations and every scheduled sweep; `realtime`: WebTransport and WebSocket hints to browsers.	Fly: `api` and `worker` are process groups of one app; `realtime` is its own app
Postgres	All durable state: servers, operations queue (pg-boss), events (NOTIFY), billing, fleet tables.	Production: Fly Managed Postgres, made by hand; staging: a Postgres 17 machine on Fly
Edge (`apps/edge`)	The only public Minecraft address: routes by hostname, answers pings for sleeping servers, wakes on join.	Its own Fly app
Runtimes	Run the game servers: Fly Machines (default), Boat sandboxes, fleet nodes with blocklyd, local Docker.	Per runtime
Archive store	Backups' durable copies, resting worlds, moves between runtimes and nodes.	Production: Cloudflare R2; staging: Tigris; local: RustFS

3 Repository and module map

Two repositories. `cubepals/cubepals`, a Bun workspace (TypeScript), holds everything below but blocklyd, which is one Rust crate in `cubepals/blocklyd`, released on its own and pinned by the control plane's image. A citation that starts `blocklyd/` is a path in that repository.

Table 3: Top-level layout.

Path	What it is
`apps/control`	The control plane: Node 22 runs the TypeScript sources directly; Hono, tRPC, pg-boss, Drizzle, Better Auth.
`apps/web`	Next.js web app; imports only **types** from the control plane (`AppRouter`) and contracts.
`apps/edge`	The edge agent (Bun, compiled) that supervises `itzg/mc-router` and speaks the edge protocol.
`cubepals/blocklyd`	The fleet's node daemon, Rust, in its own repository; `apps/control/Dockerfile` pins a release.
`apps/devtools`	A local-only inspector (`bun run devtools`); never shipped.
`packages/contracts`	Zod schemas, view types, error codes, the realtime and edge protocols: what crosses process boundaries.
`packages/db`	Drizzle schema (52 tables), 43 SQL migrations, the migrator.
`infra/fly`, `infra/terraform`	Fly app configs; Terraform modules (archive, control, dns, edge, fly-org, web) and the two environments.
`scripts/`	Operator and check scripts: boundaries, staging, smoke, `fleet.ts` and `runtimes.ts`, this document's build.
`tools/openapi`	Vendored OpenAPI specs for Fly, Boat and Modrinth, and their generated clients.
`docs/`	Design documents and operator runbooks.

Inside the control plane, layers are enforced by `scripts/check-boundaries.ts`, run in CI. An arrow means "may import".

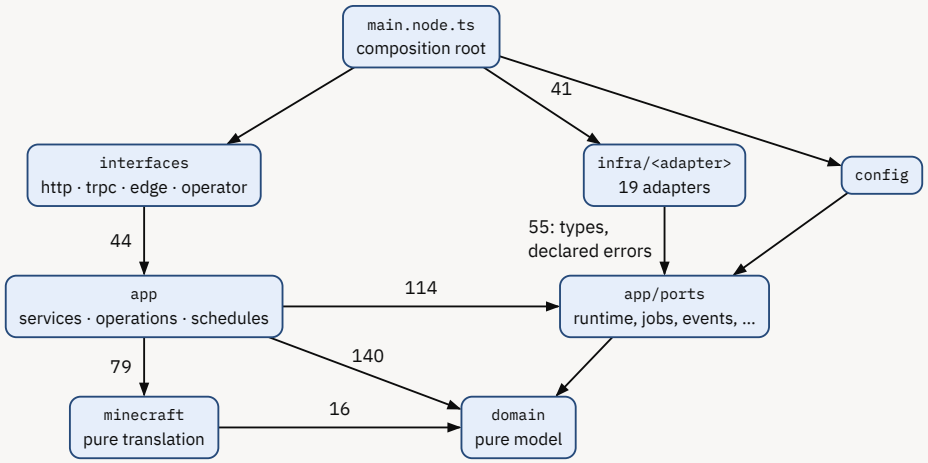


Figure 2: Control-plane layers and measured import counts (production files). No adapter imports another adapter; only `main` imports adapters. `scripts/check-boundaries.ts:133`.

Table 4: The layering rules that keep providers swappable. `scripts/check-boundaries.ts:8`.

Rule	Enforced by `check-boundaries.ts`
`domain`, `minecraft`	Pure: no packages at all; `minecraft` may use runtime **types** only.
`app`	Only `@blockly/db`, `@blockly/contracts`, `drizzle-orm`, `zod`, `node:*`; never a provider SDK.
`infra/<x>`	Only port types, declared errors and `as const` vocabularies, and itself: no cross-adapter imports. Provider SDKs only in their adapter (`@aws-sdk` in `infra/s3`, `@polar-sh` in `infra/polar`, ...).
Above the port	No runtime provider names (collected from the adapters' `provider` fields), no hostnames, no `25565`, no hostname-shaped columns.
`apps/web`	Only types from `@blockly/control`; never `@blockly/db`.

Between packages, imports run one way, toward the shared contracts and schema:

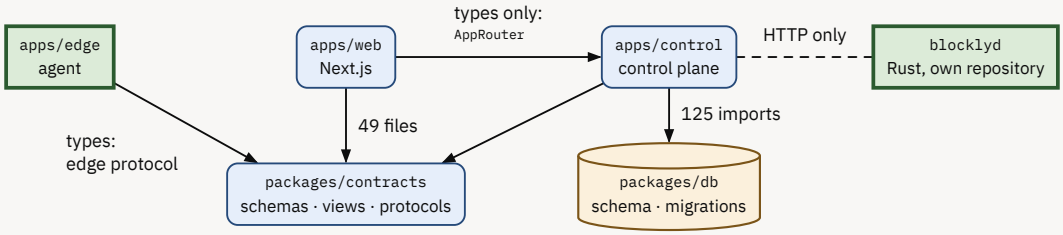


Figure 3: Package dependencies. `packages/contracts` depends only on `zod`; `packages/db` only on `drizzle-orm` and `pg`. blocklyd shares no code with the TypeScript packages: it and the control plane speak HTTP.

- blocklyd shares no code with the TypeScript side: the contract is its HTTP protocol (`blocklyd/docs/protocol.md`) and the OpenAPI documents of the pinned release, from which `apps/control/src/infra/fleet/wire.ts` takes its types.
- `infra/fleet` is the one adapter that owns database tables (`fleet_*`), read and written only by it.

> **WHY**
> The layering exists so that a provider is an adapter and nothing else: no Fly name, hostname or port number can leak upward, so swapping or adding a runtime never touches the product code. It is checked mechanically because review alone lets such leaks in one convenience at a time.

PART II

Runtimes

The one port every way of running a server goes through, how several run at once, the fleet runtime and its node daemon, what one machine holds, and how a node is chosen.

4 The MinecraftRuntime abstraction

Everything above the port speaks Minecraft and product: servers, plans, worlds, players. Everything below speaks containers, volumes and machines, and knows nothing about Minecraft (`apps/control/src/app/ports/runtime.ts:1`). A provider is an adapter, not an application change. IMPLEMENTED

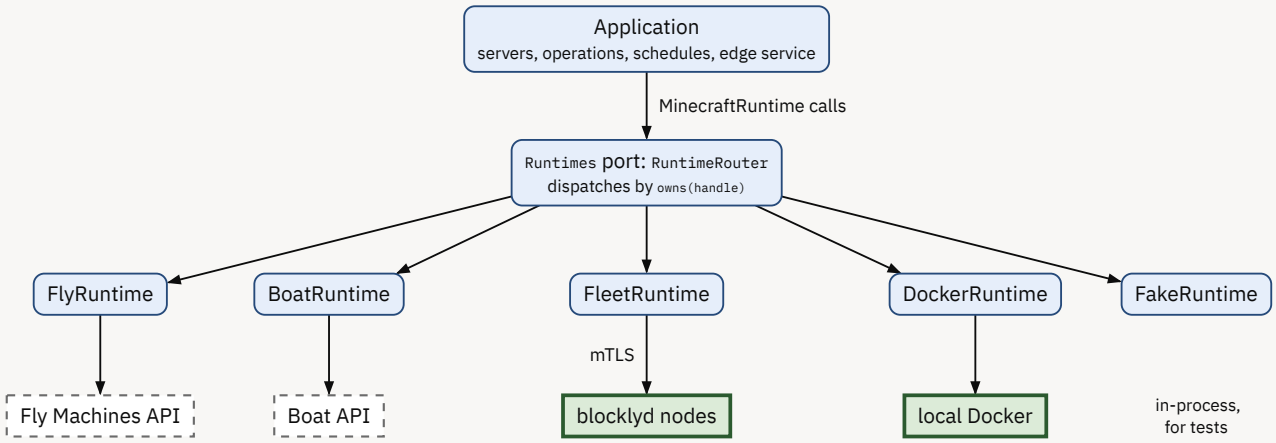


Figure 4: One port, five adapters. The router is in the path even with one runtime. `apps/control/src/app/runtimes/router.ts:131`, `apps/control/src/main.node.ts:105`.

The contract, in the parts that shape everything else:

- **Handles are opaque.** A runtime issues a branded string (`fly:v1:…`, `boat:v1:…`, `fleet:v1:…`, `docker:v1:…`); the application stores and passes it but never reads it. The binding row records the provider beside it, and `scripts/check-boundaries.ts` bans provider names above the port, so nothing up there can branch on where a server runs.
- **Intent, not steps.** `ensureProvisioned(key, placement, spec, progress)` is idempotent and convergent: called again after a crash, it finishes what exists and leaves the server started. `ProgressSink.handle()` persists a handle the moment one exists, so a crash never orphans compute.
- **Endpoints name an audience.** `endpoint(handle, port, 'edge' | 'control')`: who can reach what depends on the caller's network. `stableEndpoints` says whether a stopped server's address still reaches only it (Fly's Flycast: yes; Boat's per-run IPv6: no).
- **Declining is explicit.** A runtime may refuse `deleteSnapshot` (`RuntimeUnsupported`), leave limits null, ignore an install seed, or treat a move as a no-op where it has one place. `RuntimeFull` means "no room just then"; the owner is told that, not the provider's words.

Table 5: The port's members. `apps/control/src/app/ports/runtime.ts:192`.

Group	Members (full signatures: §A)
Lifecycle	`ensureProvisioned`, `apply`, `start` (may return a new handle), `restart` (same compute), `stop`, `forceStop`, `waitRunning`, `decommission`, `release`, `adopt`, `destroy`
World	`snapshot`, `goneSnapshots`, `deleteSnapshot`, `restore` (from a snapshot or an archive URL), `relocate` (region move, or rebuild after a lost host), `exportSnapshot`, `snapshotLifetimeDays`
Seeing	`observe`, `observeChanged` (one listing per pass), `inventory`, `isPlaced`, `sameCompute`, `exec`
Reaching	`endpoint`, `stableEndpoints`
Room and money	`serverCeiling`, `hasRoom`, `capacity`, `prices`, `tag`, `locate`
Routing	`provider`, `owns`, `ownsSnapshot`

> **WHY**
> Providers differ in how they start, stop, snapshot and fail, and in what an hour of running costs. Keeping the provider behind one port is what made Boat and the fleet possible without touching the application, and what lets them run side by side. The price is a lowest-common-denominator contract, paid for with the explicit "decline" clauses above.

CODE VS. DOCUMENTS
`docs/architecture.md` is the design the code cites by section; it was brought up to date alongside this document. Where either disagrees with the code, the code decides.

5 Several runtimes in one deployment

A deployment runs one or more runtimes at once (`RUNTIME_PROVIDERS`); every server is on exactly one, named by its binding. IMPLEMENTED. **Configured nowhere yet:** production and staging set `RUNTIME_PROVIDER=fly` only, local development `docker` (`infra/terraform/environments/production/config.auto.tfvars.example.json`).

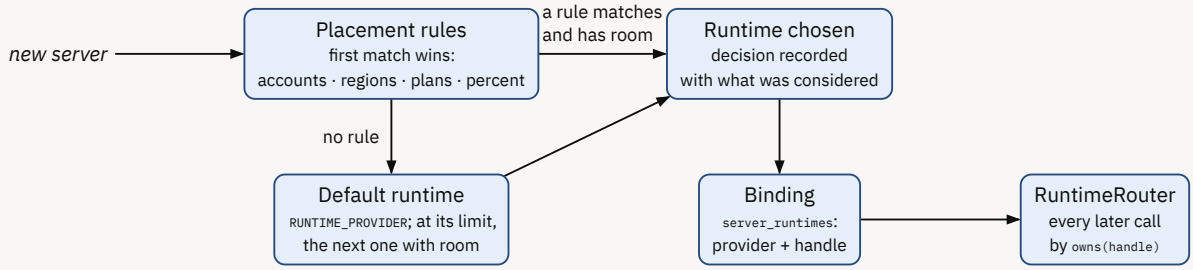


Figure 5: Placement across runtimes decides once, when a server is made. `apps/control/src/app/runtimes/placement.ts:76`, `apps/control/src/app/runtimes/service.ts:42`.

- **Rules** (`runtime_rules`) are read in creation order; the first enabled rule that matches the owner (an allowlist of accounts), the region, the plan, and the server's share wins. The share is stable: the first four bytes of `sha256(ruleId:serverId)`, as a number from 0 to 100 in steps of 0.01, against the rule's percent. A rule whose runtime is at its provider's server limit, or says it has no room, is skipped. Nothing else is weighed: no prices, no optimiser (`app/runtimes/placement.ts:10`).
- **Overflow.** If the default runtime is at its provider's limit, a new server goes to the next runtime in `RUNTIME_PROVIDERS` with room (`overflow: …`).
- **First-start fallback.** A server a rule sent to another runtime, that never started, whose runtime says `RuntimeFull`, moves once to the default runtime (`fell_back`), and what the first runtime began is destroyed (`apps/control/src/app/runtimes/service.ts:109`). One the default overflowed has no fallback, since the default is full. After a first start there is no fallback: a start without room puts the server back to sleep.
- **Region maps.** `RUNTIME_REGION_MAP` is the default runtime's. Every other runtime places only the regions its own `<NAME>_REGION_MAP` names, and boot refuses one without (`apps/control/src/config/schema.ts:288`).
- **Moves between runtimes** are only an operator's (`bun scripts/runtimes.ts move <id> --to fly`). The relocation sweep makes them at the server's next quiet moment, through the archive store: stop, archive copy (size checked, `level.dat` found), `adopt` on the target, restore, rebind in one transaction, then the old runtime's copy is destroyed. A failed move leaves the server where it was (`move_failed`). A resting server moves by its binding alone.
- **Foreign bindings.** Dropping a runtime from `RUNTIME_PROVIDERS` makes its servers `foreign`: refused by every handler, not migrated. It is not an off-switch.

WHY
Decide once, record why, never move a server because a rule changed: turning a canary off strands nothing, and where a server runs is explained in one place (`apps/control/src/app/runtimes/service.ts:42`). Owners are never asked: where a server runs is Blockly's decision (AGENTS.md's core rule), and it is visible to operators in `runtime_decisions`.

6 FleetRuntime

The fleet is Blockly's own runtime: Minecraft servers on Linux machines Blockly rents, each running blocklyd. There is no cluster: no quorum, no consensus store, no Kubernetes or Nomad, no service mesh, no distributed filesystem, no live migration, and no automatic failover (`docs/fleet.md:3`). IMPLEMENTED in code; CANARY ONLY in use: reached only when an operator adds `fleet` to `RUNTIME_PROVIDERS` and sends servers there. Never run on independent real hosts.

One machine is a whole fleet. The same runtime, tables and code paths serve one node or many.



Figure 6: The fleet with one node (left) and with several (right). Nodes never talk to each other; worlds move only through the archive store.

Table 6: What a second node changes. Nothing else does.

	One node	Several nodes
Placement	One choice: the node, or `RuntimeFull`.	Filter and score every node (§9).
A full node	A start waits for room (`RuntimeFull`, back to sleep).	A sleeping server moves to a node with room, then starts.
Operator moves, drains	Refused as `RuntimeFull` before anything stops: the server stays where it is, running if it ran.	Work: through the archive store, with a new epoch.
The host is lost	Servers wait, or are rebuilt from the archive store once another node exists.	Rebuilt on other nodes from their last uploaded snapshots (§24).
Backups	Still leave the machine (archive store): a lost host loses at most the play since the last upload.	The same.

What FleetRuntime owns, beside the port's methods:

- **The ledger** (`fleet_placements`): per server, its node, epoch, state (`placing`, `placed`, `displaced`, `released`), size, desired power and when it last changed (`docs/fleet.md:152`).
- **Epochs and fences** (§14), **moves and rebuilds** (§22, §24), **two-step snapshots** (§20), and **running admission**: only servers that run, start or are being made hold memory (§8).
- **The node endpoint** (`FLEET_NODE_LISTEN`, 8443, on the `api` role): enrollment, heartbeats, renewal; and **upkeep** on the `worker` role each minute.
- **Cost**: a node may carry its monthly price as a label (`monthly_cost_cents`); a server's share by memory is reported as its storage-month price (running hours cost 0: the node is paid for either way).

7 blocklyd

One static Rust binary per node (about 11,400 lines; `cubepals/blocklyd`, under FSL-1.1-ALv2), running as root beside Docker. It owns what exists on its host and decides almost nothing. IMPLEMENTED

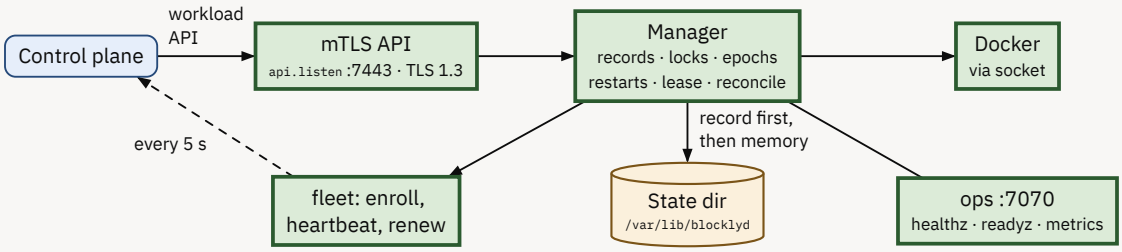


Figure 7: blocklyd's parts. `blocklyd/src/manager.rs` (3,296 lines) is the core; `blocklyd/src/api/mod.rs:460` the routes; `blocklyd/src/fleet/heartbeat.rs`.

Table 7: What blocklyd owns. `blocklyd/README.md:15`, `blocklyd/src/manager.rs:4`.

blocklyd owns	blocklyd does not own (and who does)
Containers it made, labelled with its deployment and node; it never touches others.	Where anything runs: the control plane's placement. It "never chooses where anything runs".
Host ports (42000–42999, quarantined 600 s after release).	Whether a server should run: only requests and its lease-gated restarts move power.
Data directories, local snapshots, trash, the spool.	Backup schedules, uploads' retries, retention: FleetRuntime's upkeep.
Its records (`workload.json`, written before memory) and a copy in each container's labels.	The desired state: Postgres.
Restarting a failed workload (on-failure, 3 tries, backoff 0.5 s doubling to 10 s), only under a lease; resuming after a reboot.	Declaring a host lost, failover: an operator.

blocklyd owns	blocklyd does not own (and who does)
Its identity: its own key; certificates from the control plane's CA.	Issuing certificates; DNS and routes (the edge); billing; provisioning machines; the firewall.
Enforcing host policy on every spec: image allowlist, non-root user, limits.	Anything Minecraft: quiescing (`save-off`), RCON, what to exclude from a move are the caller's.

- **Protocol.** HTTP/JSON over mutual TLS (TLS 1.3 only, client certificates with `clientAuth`, the client's DNS name in an allowlist, no 0-RTT), 19 operations on 16 paths, one error shape, features announced instead of versions (§C). Mutating verbs run detached, so a dropped connection never leaves half a change.
- **Every verb is idempotent and checks an epoch rule**: `Exact` to run or copy the current copy, `Teardown` to stop or delete any copy, `Place` to create or replace (`manager.rs:122`).
- **Startup.** Lock the state dir, check it can give data to the workloads' user (refusing to start otherwise, since `105f647`), enroll if needed, reconcile against Docker, then serve and beat.

WHY

Why a daemon at all, rather than the control plane driving Docker remotely: Docker's API over TCP is root on the host for whoever holds the credential, and it knows nothing of epochs, leases or link-proof walks. A small daemon with a narrow protocol is the security boundary and the place where safety rules (epochs, the lease, the walk) are enforced next to the data. **Why Rust:** one static binary, about 9 MB resident, nothing to install on a host but a kernel and Docker.

8 One machine, many servers

How much of a node servers may use, and why "20 servers on a 64 GB machine" is a ceiling, not a promise. IMPLEMENTED (the ledger); density RESEARCH ONLY (never measured).

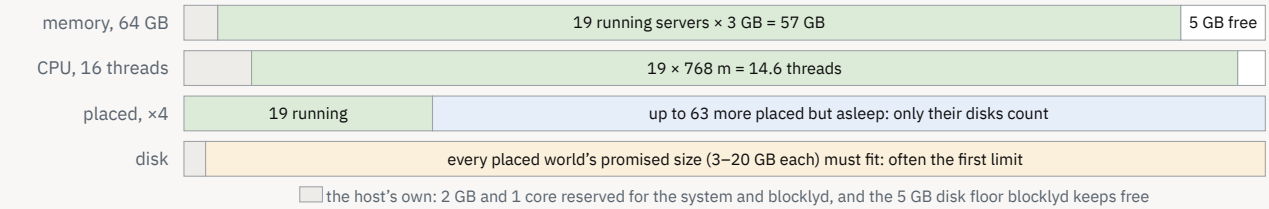


Figure 8: A 64 GB, 8-core/16-thread node with 3 GB servers, at the defaults: host reserve 2 GB and 1 core (blocklyd), 250 m of CPU per GB, placed memory ≤ 4 × allocatable. Memory alone fits 20 running; the CPU ledger stops at 19. `apps/control/src/infra/fleet/placement.test.ts:191` checks the same arithmetic: 19 running, 78 asleep on a 60,000 MB node.

- **Running memory is never overcommitted.** A JVM with a pre-touched heap (`AlwaysPreTouch`) uses what it is given from boot; the node refuses a start past allocatable × `memory_overcommit` (1.0). A 3 GB server is a 3 GB container: a 2 GB heap plus about 650 MB the JVM needs outside it, plus page cache (`apps/control/src/minecraft/runtime-spec.ts:78`).
- **Sleeping servers hold no memory** (running admission): placed servers may total `FLEET_MEMORY_OVERCOMMIT` (4) × allocatable, because a server is asleep most of the time. A waking server on a full node moves first. Setting the overcommit to 1 restores the old ledger, where every placed server held its memory.
- **CPU is bookkeeping, not a quota.** The ledger counts 250 m per GB, but the control plane sends no CPU limit (`cpuMillis` is unset, `apps/control/src/infra/fleet/fleet-runtime.ts:223`): containers get a CPU **weight** by size, so a busy server can take an idle neighbour's CPU, and neighbours share the pain under contention.

CAVEAT

Why "20 servers" is not guaranteed. (1) The host reserve is a default, not a measurement: 2 GB, where kernel, page cache, Docker and local snapshots may take 4–8% of RAM. (2) A 16-thread box binds on CPU before memory, a box with fewer threads much sooner (64 GB on 8 threads: 9 servers). (3) Disk often fills first: placed worlds keep their promised size. (4) Minecraft's tick is single-threaded and players, plugins and exploration vary by an order of magnitude; no density test (MSPT, steal, throttling with many containers) has been run on a candidate machine. (5) Running admission is statistical: a busy evening can find the node full and move a waking server.

9 Placement

Choosing a node: deterministic and pure (`apps/control/src/infra/fleet/placement.ts`), on the ledger, never on what nodes report, under a Postgres advisory lock per region so two placements can't take the last room. IMPLEMENTED

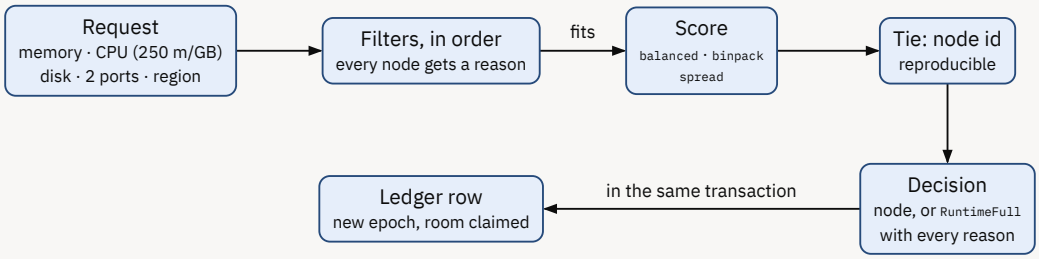


Figure 9: Placement. A refusal reads like `memory: 900 MB free after headroom, needs 3072`.

Table 8: Filters. `apps/control/src/infra/fleet/placement.ts:102`.

#	Filter (the first that fails is the node's reason)	Refusal begins
1	Not a move's source; the one node an operator named, if any	`excluded, not the node asked for`
2	Lifecycle `active`; not quarantined; health `healthy`	`lifecycle …, quarantined…, health …`
3	In the server's fleet region; has the protocol features needed	`region …, lacks …`
4	Memory to run **now**: allocatable − running − `FLEET_HEADROOM_MB` (0)	`memory: … free after headroom`
5	Memory to **stay**: allocatable × `FLEET_MEMORY_OVERCOMMIT` (4) − placed	`memory: … left to place servers in`
6	CPU now: (threads × 1000 − reserved) × `FLEET_CPU_OVERCOMMIT` (1) − running	`cpu: …m free`
7	Disk free above the node's floor; disk not yet promised × `FLEET_DISK_OVERCOMMIT` (1)	`disk: …`
8	Host ports	`ports: …`

Policies (`FLEET_PLACEMENT`) score what is **placed**, running or not, against what may be placed, because a placement lasts as long as its world stays while what runs changes by the hour:

- `balanced` (default): best fit on memory among nodes under the CPU pressure line (`FLEET_CPU_PRESSURE`, 0.8); past it, the least CPU-pressed node.
- `binpack`: the fullest node that fits. `spread`: the emptiest.

Not built: cost-aware placement. Nothing weighs prices: not between nodes, not between runtimes ("no prices, no optimiser", `apps/control/src/app/runtimes/placement.ts:10`). Moving servers between nodes over time (rebalancing) is likewise DEFERRED.

> **WHY**
> Placement on the ledger rather than on observations means a burst of placements can't overcommit a node between two heartbeats, and a decision can be replayed from its inputs. The node stays the final arbiter: if it refuses a start the ledger allowed (it runs a copy the ledger doesn't know), the claim is given back and the server moves after all.

PART III

Fleet state and safety

Who owns which truth, how it converges, how nodes join and are watched, and the three mechanisms that keep a server from running twice: epochs, fences and the execution lease. Then a server's life, its start, and how sleep differs between runtimes.

10 State ownership

Every fact has one owner. Others hold copies, and a copy is never the reason anything happens. IMPLEMENTED

Table 9: Who owns what. Sources: `docs/fleet.md:152`, `apps/control/src/infra/fleet/registry.ts`, `blocklyd/src/store.rs:1`, `apps/control/src/app/operations/schedules.ts:598`.

Fact	Owner, and where	Copies	When they disagree
What a server should be: status, plan, size, region, settings, its runtime	Control plane, Postgres: `servers`, `server_runtime`, operations	The runtime's spec of it; the node's workload record	The control plane wins: `drift` re-applies a spec when nobody plays (§11).
Which node a fleet server lives on, and its epoch	FleetRuntime, Postgres: `fleet_placements`, `fleet_placement_history`	The node's record (`epoch`, `superseded_by`) and its container labels	The higher epoch wins; an older copy is fenced (§14). A restored database adopts a newer node epoch for its own placement (`epoch.adopted`).
What exists on a host: containers, ports, data, local snapshots	blocklyd, the node's disk: `/var/lib/blocklyd/workloads/<id>/workload.json`	`fleet_observations` (last report, written on change); Docker labels	The node's report replaces the observation on every heartbeat. Records lost from disk are rebuilt from labels (`record_rebuilt`).
Whether a node is alive	Nobody: health is derived from heartbeat age in database time	`fleet_nodes.last_heartbeat_at`	Health is a reading, not a verdict (§13).
Whether a host is gone	The operator, by `lost --fenced-by` (`fleet_nodes.lifecycle`)	—	Never derived from silence. A node that returns while lost is fenced, not trusted.
A world's durable copy	The archive store (S3-compatible), `fleet_archives` or the runtime's snapshots	The node's local snapshot copy	The store's copy is the one a rebuild uses; local copies are trimmed once a newer upload lands.
Who is online	The server itself, read over RCON each minute (`presence-sync`)	`presence` rows	A server that doesn't answer is asked again; empty presence just after a restart means "not read yet" (5 min grace).

WHY

A control plane that also held "what is on each host" would have to be right about hosts it can't see. A node that decided what should run would need the whole picture. Splitting desired (control plane) from existing (node) lets either side rebuild itself from the other: a node from its own records and Docker's labels, the fleet from Postgres, without a consensus system.

11 Desired state and reconciliation

Nothing is reconciled by a single loop. Three owners each converge their own part, on their own clock. IMPLEMENTED

- **Application** (worker role, pg-boss singletons, `apps/control/src/infra/pg/jobs.ts:266`): `reconcile` each minute reads what changed at the provider since the last pass (a full read hourly) and marks crashes; `drift` re-applies a spec that changed, five at a time, only while nobody plays; `relocations` moves misplaced or lost-host servers, three at a time; `idle-check` stops servers idle past their plan; `backup-schedule` hourly at :23.
- **FleetRuntime** `upkeep` each minute (`fleet-runtime.ts:1420`), in order: health changes, a probe of silent nodes, placements a report settled, placements their node never made undone, tidying superseded copies, uploads of local snapshots (four at a time, five tries), deleted copies, trimming local copies, and pruning events older than 90 days. Each failure is logged and the next task runs. No loop pushes desired specs to nodes: a node is only ever told something by a call the application made through the port.
- **blocklyd**: a Docker event stream plus a full resync every 30 s; disk usage every 300 s; stats every 15 s (`blocklyd/src/config.rs:354`). Its records are written to disk before memory, so a restart rebuilds it in about a second.

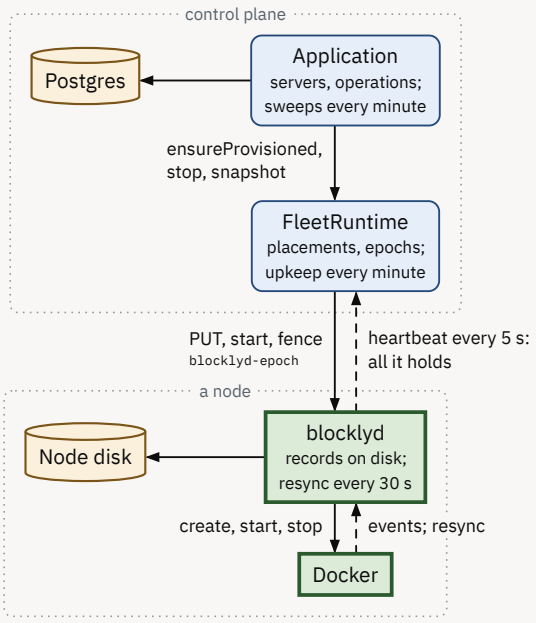


Figure 10: Three owners, three loops; FleetRuntime keeps its tables in the same Postgres. Solid: calls; dashed: reports. Sweeps: `apps/control/src/infra/pg/jobs.ts:266` blocklyd intervals: `blocklyd/src/config.rs:346`.

WHY

Level-triggered loops forgive lost messages: a missed heartbeat or a dropped event is repaired by the next full pass, and every pass is idempotent. The cost is latency (up to a minute for most sweeps), which a hosting product with human-scale actions can afford.

12 Enrollment

How a host becomes a node. **IMPLEMENTED**

- The control plane names the node (`<node-id>.nodes.<deployment>.fleet`); the CSR's subject is ignored. The identity is the certificate, never an address. TLS 1.3 only, both ways.
- Two hosts with the same `/etc/machine-id` hash are recorded (`node.machine_id_reused`): a cloned image. Two daemon sessions beating as one node quarantine it (`node.duplicate_identity`).

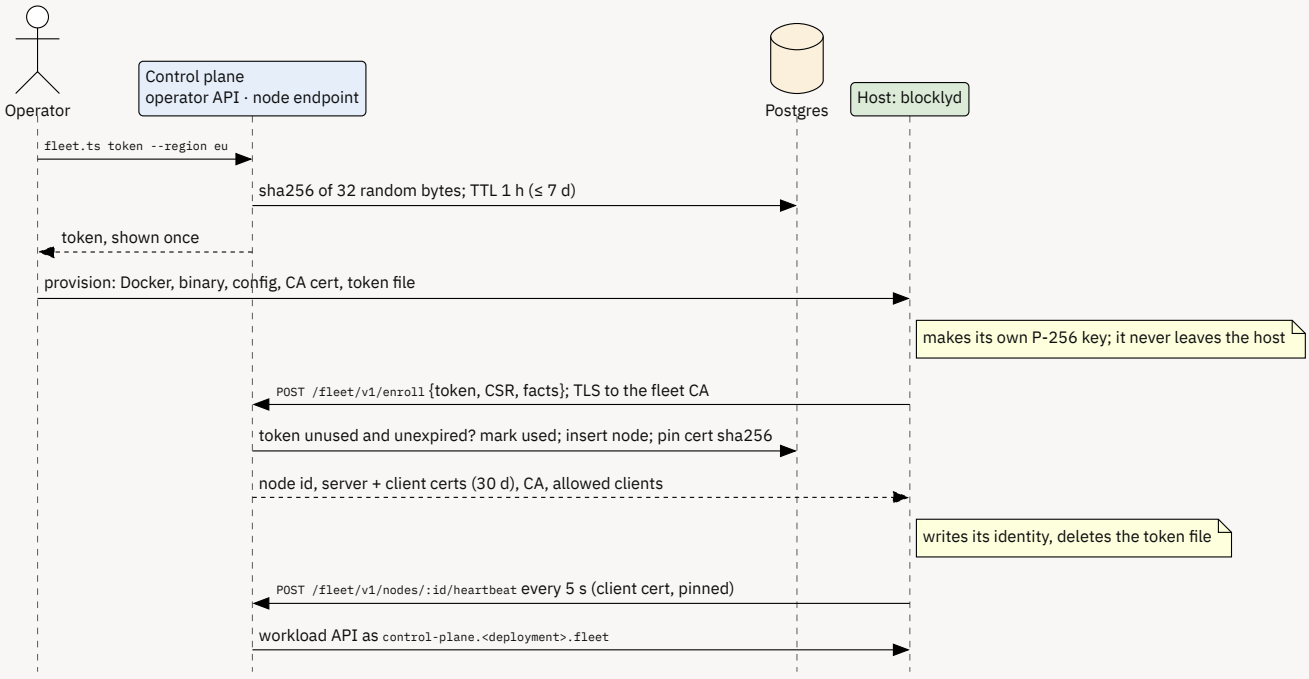


Figure 11: Enrollment. `apps/control/src/infra/fleet/node-endpoint.ts:17`, `apps/control/src/infra/fleet/registry.ts:168`, `blocklyd/src/fleet/enroll.rs:30`.

- Renewal: certificates last `FLEET_NODE_CERT_DAYS` (30). Within `FLEET_RENEW_DAYS` (10) of the end, heartbeat answers carry `renew`; the node makes a new key, calls `…/renew` with its current certificate, and swaps its files by one rename. The pin moves on the first beat with the new one.
- A node whose identity was lost re-enrolls under its own id with a token bound to it (`token --node`), and adopts its own containers (`node.reenrolled`).
- An answer lost on the way (a network drop, or a disk that refused the identity) is asked for again: until it is enrolled the node keeps one key on disk, across restarts too, and the control plane answers a spent token again, before its expiry, for the key that spent it (`node.enrollment_repeated`, `blocklyd/src/fleet/enroll.rs:8`, `apps/control/src/infra/fleet/registry.ts:284`). Any other use of a spent token is refused.

CAVEAT

A node that lost that key before its answer arrived (its state directory wiped) cannot prove it is the one the token enrolled: an operator mints a new token, and the half-made node is retired by hand. Nothing is compromised.

13 Heartbeats and node health

Every 5 seconds (`heartbeat_seconds`, `blocklyd/src/config.rs:119`) a node sends everything it holds: capacity, every workload's state and epoch, issues, addresses. The answer carries its lifecycle, fences, the lease, and `renew` when due (`apps/control/src/infra/fleet/wire.ts:234`). A node that was cut off, restarted or rebooted resynchronises on its first beat. **IMPLEMENTED**

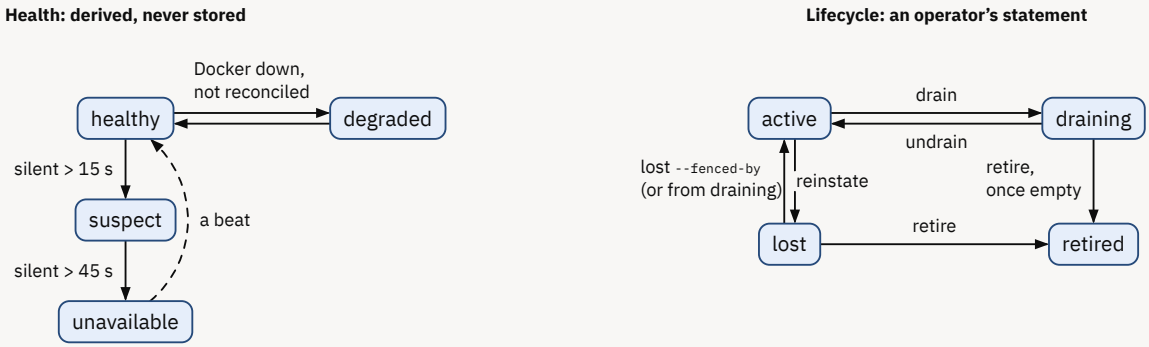


Figure 12: Health and lifecycle are separate. Thresholds: `apps/control/src/infra/fleet/health.ts:17` (`FLEET_SUSPECT_SECONDS`, `FLEET_UNAVAILABLE_SECONDS`); lifecycle actions: `apps/control/src/infra/fleet/operator-api.ts:116`.

Table 10: What each health means. `apps/control/src/infra/fleet/health.ts:32`.

Health	What it means	What happens automatically
healthy	Beat within 15 s; Docker answers; reconciled since.	Eligible for placement.
degraded	Beating, but Docker doesn't answer or the node hasn't reconciled since.	No new placements. Servers keep running.
suspect	Silent 15–45 s, or longer while the endpoint itself has been up less than 45 s.	No new placements. Its last report is kept, so a missed beat doesn't flap a server's state.
unavailable	Silent over 45 s, or never beat.	Its servers read `unknown`. **Nothing moves and nothing restarts elsewhere.**

WHY

Unavailable is not dead. Silence is a fact about the network between two machines, not about the host. A partitioned node keeps serving its players (§25); rebuilding its servers elsewhere on a timer would run two copies and fork worlds. So `lost` is never derived: an operator declares it after fencing the host. `confirmLost` refuses only while the node still beats (healthy or degraded) unless forced, so it is allowed from 15 s of silence on (`apps/control/src/infra/fleet/registry.ts:860`): the operator, not a timer, carries the judgement. The endpoint's own uptime counts too: heartbeats sent while nothing listened aren't the nodes' fault (`fleet_endpoints`).

14 Epochs and fencing

Every new home of a server gets a **placement epoch**: a number that rises in the transaction that changes the placement and is never reused (`nextEpoch`, `fleet-runtime.ts:561`). Every mutating request to a node carries it in `blocklyd-epoch`. **IMPLEMENTED**

- A node refuses a request older than the copy it holds (`stale_epoch`), refuses to run a copy for another epoch, and refuses everything but teardown on a copy a newer epoch superseded.

- **A fence** says “this workload is at epoch N”. The node marks an older copy `superseded_by N`, switches off its restarts, stops it, keeps its data. Late or repeated fences change nothing. Fences arrive in every heartbeat answer and, during a move, directly (`POST …/fence`).
- A superseded copy is never started again, by anyone, even after a reboot. Since `eacb334` the mark is held in memory first, so a disk that refuses the record (full, read-only) still stops the copy and keeps it from resuming; the failed write is reported.

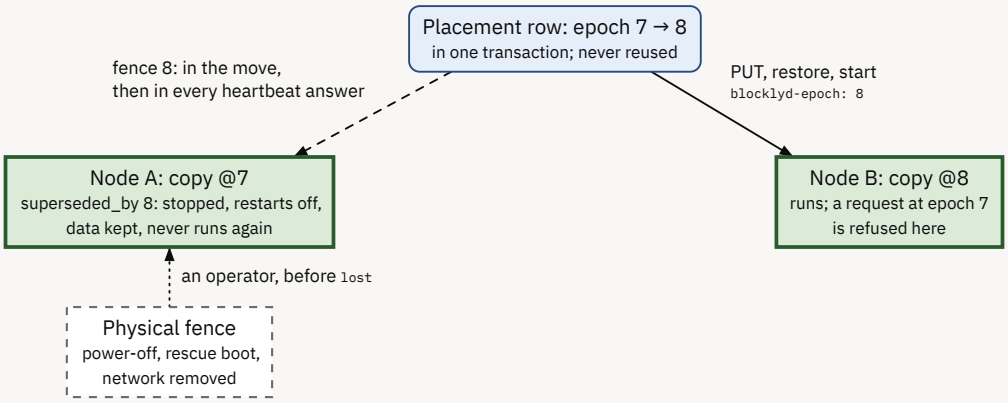


Figure 13: Logical fencing needs the node to hear the fence; physical fencing doesn’t. An operator does the second before declaring a host lost.

Table 11: Two kinds of fence. `docs/fleet.md:209`.

	Logical fence (epochs)	Physical fence (operator)
What	The node refuses old epochs and stops superseded copies.	The host stops: powered off in the provider’s console, rescue boot, network removed.
Needs	The node to be reachable, or to hear its next heartbeat answer.	Nothing from the node.
Protects against	A stale request, a late retry, a returning node, a restored database.	A partitioned node still serving its players.
Status	IMPLEMENTED	IMPLEMENTED as a required statement (`--fenced-by`); no provider API does it (DEFERRED).

- **A copy of no placement at all** (its server was destroyed while its node was unreachable, or the database was restored from before it existed) is fenced one epoch past its own and reported (`copy.orphaned`). The application’s hourly `orphans` sweep destroys a copy whose server was purged (its data goes to the node’s trash for 24 hours); one whose server the database doesn’t know is kept and logged each time it is seen, for an operator, since after a restored database that is every server made since (`apps/control/src/app/operations/schedules.ts:808`).
- **A restored database** behind a node: the node’s newer epoch is adopted for its own placement (`epoch.adopted`) and flagged for any other (`epoch.conflict`).

> **WHY**
> Epochs make the safety property local: a node can decide alone whether a request is current, with no clock and no quorum. That is what lets the fleet have no consensus store. What epochs can’t do is reach a node that can’t be reached, hence the operator’s physical fence.

15 Execution leases

blocklyd is the restart authority on its host; Docker’s restart policy is always `no` (`blocklyd/src/manager.rs:762`). It restarts a workload that failed, per the spec’s policy (`on-failure`, 3 tries with backoff), and resumes after a host reboot what was running, **only while it holds an execution lease**. IMPLEMENTED

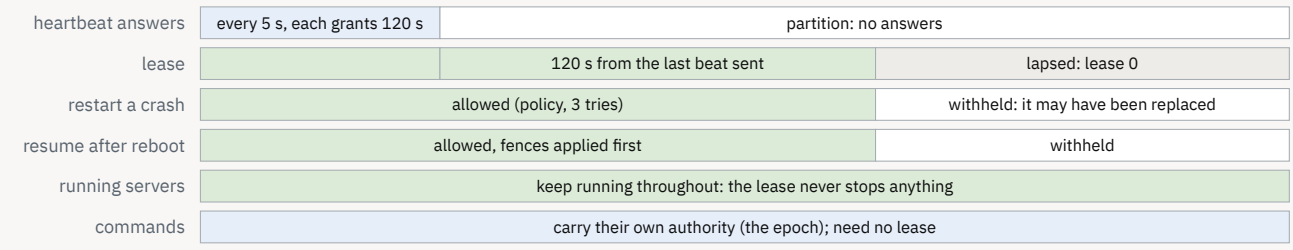


Figure 14: A partition starts after the fourth answer. Lease measured on `CLOCK_BOOTTIME` from when the beat was **sent**, so a paused VM or a slow answer can’t stretch it. `blocklyd/src/manager.rs:2513`, `apps/control/src/config/load.ts:91`.

- The answer grants `FLEET_LEASE_SECONDS` (120); a node held `lost` gets 0. Without an answer naming one, `restart_requires_contact_seconds` (120) applies.
- The heartbeat that grants a lease applies its fences first, so a server rebuilt elsewhere during an outage never resumes here.
- A restart of blocklyd alone (same boot id) resumes nothing: what died while it was down is reported, not restarted.

WHY
A node that hasn't heard from the control plane for a lease may already have been replaced: restarting its copy could fork a world. But stopping running servers when the lease lapses would turn every control-plane outage into a fleet-wide one. So the lease withholds only what the node would do unasked. The consequence is §25's split-brain window.

16 Workload lifecycle

Two state machines, one per owner. The application's is the product's (what the owner sees); blocklyd's is the host's (what the container is doing). IMPLEMENTED

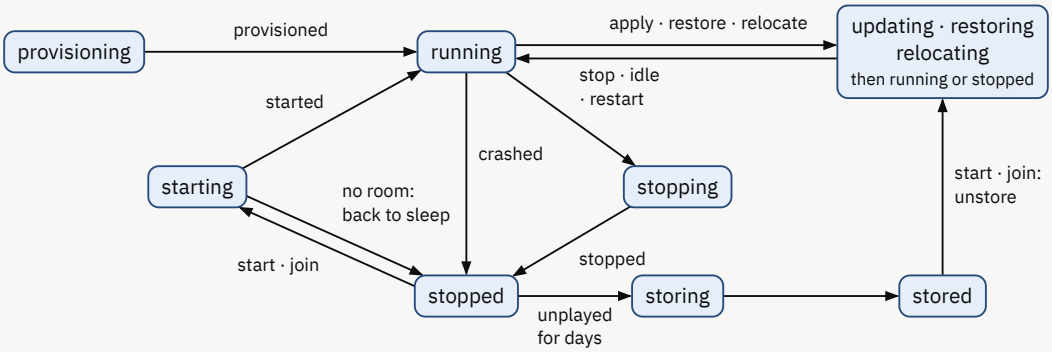


Figure 15: The server's lifecycle, simplified (`apps/control/src/domain/server/lifecycle.ts:212`). Not drawn: any phase can end in `failed`, and `retry` resumes the phase it failed in; a restart's stop half goes from `stopping` back to `starting`; `deleted`, then `purged`. A start that finds no room puts the server back to sleep (`refused`, idle) instead of failing (`apps/control/src/app/operations/handlers.ts:435`).

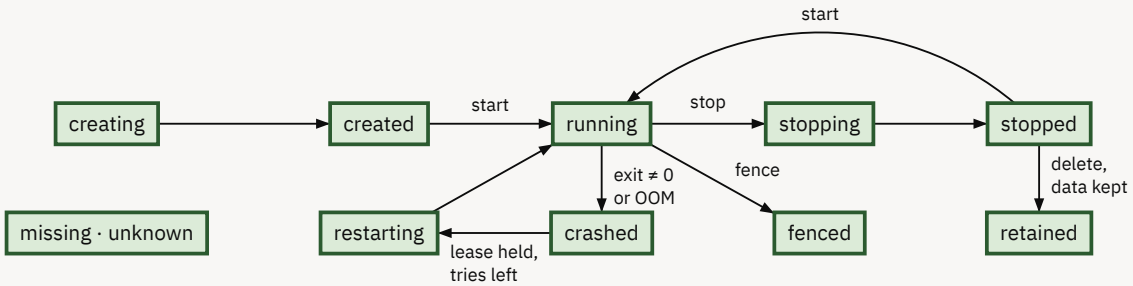


Figure 16: A workload on a node (`WorkloadState`, `blocklyd/src/protocol.rs:495`), derived from blocklyd's record and Docker's view (`blocklyd/src/manager.rs:3217`). A superseded copy at rest reads `fenced`; a stopped or crashed copy waiting to resume after a reboot reads `restarting`; `missing`: the container vanished; `unknown`: Docker doesn't answer.

17 The startup path

What happens between a press of Start (or a player joining a sleeping server) and a playable world, on the fleet. Other runtimes follow the same application path and differ inside `ensureProvisioned` (§18). IMPLEMENTED for the fleet half.

Where it can go wrong, and what the owner sees:

- **No room on the node:** the sleeping server moves first to a node in its region with room (its world through the archive store), then starts (`placement.moved_for_room`). No room anywhere, or a world too big to copy: `RuntimeFull`, and the server goes back to sleep with a plain-words reason.
- **Never started before, and its runtime is full:** a server a rule sent to this runtime falls back once to the default runtime, and it is recorded (`handlers.ts:278`); on the default, it waits for room. CANARY ONLY for the fleet.
- **The node refuses a start the ledger allowed** (it runs a copy the ledger doesn't know): the claim is given back and the server moves after all.
- **A pack that learns on first boot** starts again on what it learned (`boot.run`).

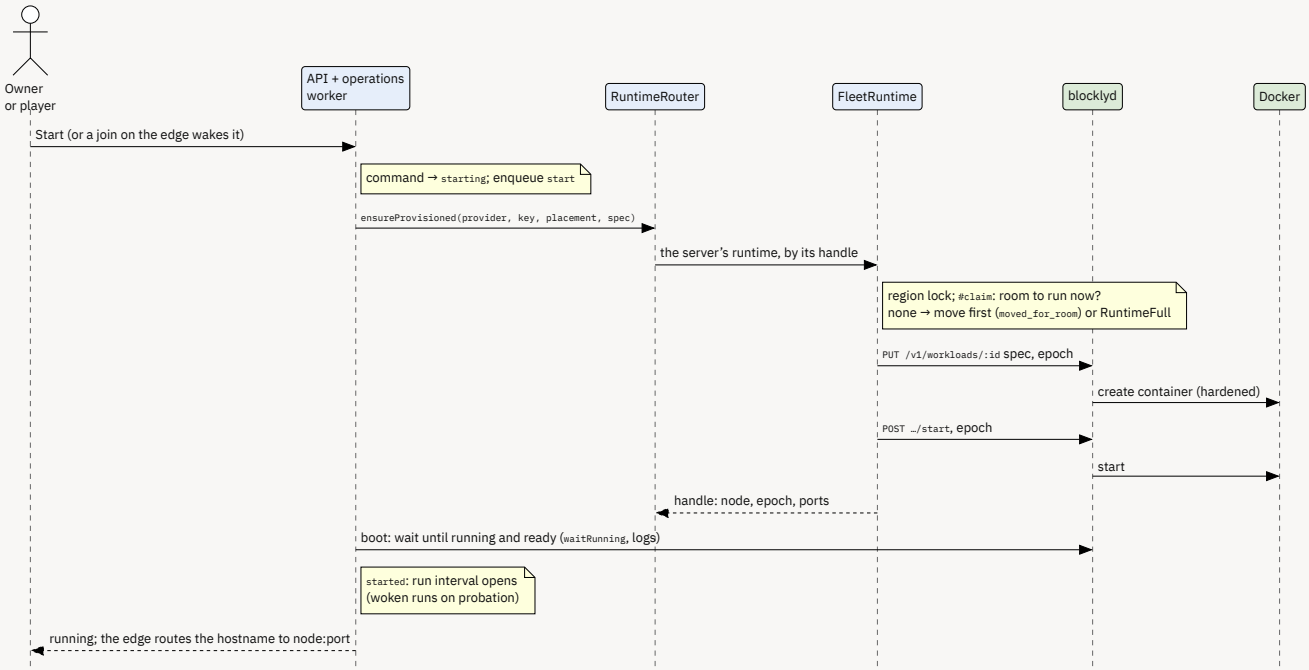


Figure 17: Startup on the fleet. `apps/control/src/app/operations/handlers.ts:291` (bringUp), `apps/control/src/infra/fleet/fleet-runtime.ts:527` (ensureProvisioned).

18 Sleep and wake on Boat, Fly and the fleet

"Asleep" means the same thing to the owner everywhere: stopped, world kept, a join wakes it. What it costs, and what is kept, differs per runtime. Details of each runtime are in §27–§29.

Table 12: Sleep and wake per runtime. `apps/control/src/infra/fly/fly-runtime.ts:397`, `apps/control/src/infra/boat/boat-runtime.ts:43`, `apps/control/src/infra/fleet/fleet-runtime.ts:1792`.

	Fly	Boat	Fleet
Asleep, it keeps	Its stopped Machine, volume, app and Flycast address	Boat's own snapshot of the sandbox's disk; no compute, no address	Its world on the node's disk, its node address and host ports; no memory claim
Waking needs	The Machine to start on its host	A Boat start from the plan's quota (per minute, hour and day), then a new machine	Room on its node; else a move to a node in the region with room
Address after wake	The same (`stableEndpoints`)	A new IPv6 each run; the edge routes to `0.0.0.0:0` while asleep	The same while placed
No room	A host with no room is `RuntimeFull` → back to sleep; Fly never starts it elsewhere	`RuntimeFull` when starts are spent → back to sleep	`RuntimeFull` after every node → back to sleep
Wake time	Not measured by Blockly	About 34 s from join to running; the edge waits 25 s, so a first join sees "starting"	Only prototype numbers, on one VM (RESEARCH ONLY)
Kept asleep	The volume and the stopped root filesystem	Only the archive	Its disk on the node

WHY

The application decides **when** a server sleeps (idle past its plan, a woken run nobody joined within five minutes, `schedules.ts:776`) and the runtime decides **what sleeping keeps**. That split is what lets a fleet server give back its memory while a Fly server keeps its Machine.

PART IV

Data

Where worlds live, how they are backed up and restored, and how they move: between nodes, off a draining node, away from a lost one. Then the one failure the design does not prevent by itself.

19 The storage model

A world has one **hot** copy, where its server runs, and **durable** copies that outlive the machine. Nothing in Blockly shares a hot copy between machines: no distributed filesystem, no network block device for worlds. IMPLEMENTED

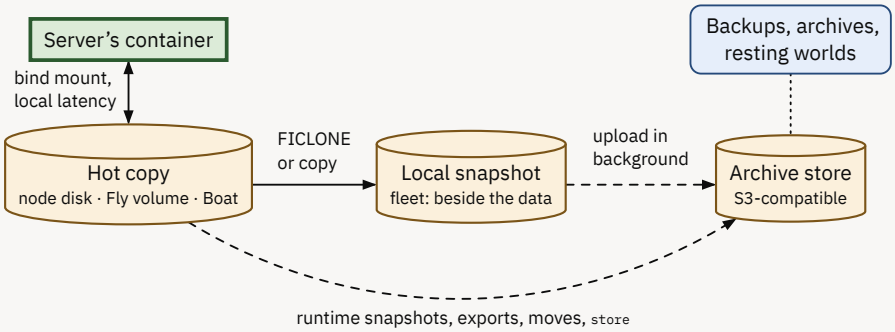


Figure 18: Hot and durable copies. The local snapshot tier exists only on the fleet; other runtimes snapshot with their provider (§27).

Table 13: Storage tiers. `blocklyd/src/store.rs:3`, `blocklyd/src/tree.rs:330`, `blocklyd/src/transfer.rs:95`, `apps/control/src/domain/account/entitlements.ts:150`.

Tier	What	Why	Status
Hot, fleet	`/var/lib/blocklyd/workloads/<id>/data` on the node's own disk, bind-mounted, owned by the workloads' uid (1000:1000).	Minecraft's region saves are fsync-heavy and latency-bound; provider block volumes have millisecond fsync.	IMPLEMENTED
Local snapshot	`snapshots/<id>/data` beside it, never mounted. `FICLONE` per file where the filesystem shares blocks (XFS reflink, btrfs), else a full copy.	A backup costs the player seconds; restoring on the same node needs no download.	IMPLEMENTED reflink path not run on a real reflink disk
Archive store	Gzip tarballs (`./`-relative paths, fastest level). Every archive's sha256 is recorded as it is written (the fleet's own copies, and the application's `backups` since migration 0044), and a node checks it as it downloads. Presigned URLs (30 min); nodes hold no credentials. One PUT carries at most 5 GiB less 5 MiB; a larger archive goes as a multipart upload, a presigned URL for each part.	Survives the machine; one format every runtime reads and writes.	IMPLEMENTED parts
Resting worlds	A server unplayed for its plan's `storeAfterIdleDays` (Free 14, Plus 30) is `stored`: its world in the archive store, no compute, no disk.	Idle worlds cost object storage, not disks. A join or start brings it back.	IMPLEMENTED
ZFS	`zfs snapshot` and `zfs send` streams, evaluated.	Needs an out-of-tree kernel module on every node; nothing could be measured honestly in the test environment.	RESEARCH ONLY

- **The disk floor.** blocklyd keeps `min_free_disk_mb` (5 GB) free: a create, an export's spool, a snapshot and a restore are refused below it with `507 insufficient_disk`. A restore from a URL is refused too when the size its download announces wouldn't fit above the floor, and is stopped if the floor is reached while it downloads or unpacks (`blocklyd/src/manager.rs:1532`). Placement admits disk for the world when it places the server (§9), which covers the normal case.
- **A world's size is measured, not enforced.** `storage.sizeGb` is checked against usage and reported as an issue (`over_storage`); there is no quota on the node (DEFERRED).
- **Trash.** Deleted data and data a restore replaced go to `trash/` for 24 hours, so a restore can be undone by hand; it is purged hourly.

> **WHY**
> Worlds on local disks make a node's loss a data question: what was played since the last uploaded snapshot is on that host only (§24). The alternative, network storage, would make every save as slow as the network and every node depend on a storage cluster. For servers that sleep most of the day, a two-tier model (fast local disk, durable object store) costs least and fails in the most understandable way.

20 The backup pipeline

Backups are the application's; runtimes only take and keep snapshots. IMPLEMENTED, with the fleet's two-step snapshot.

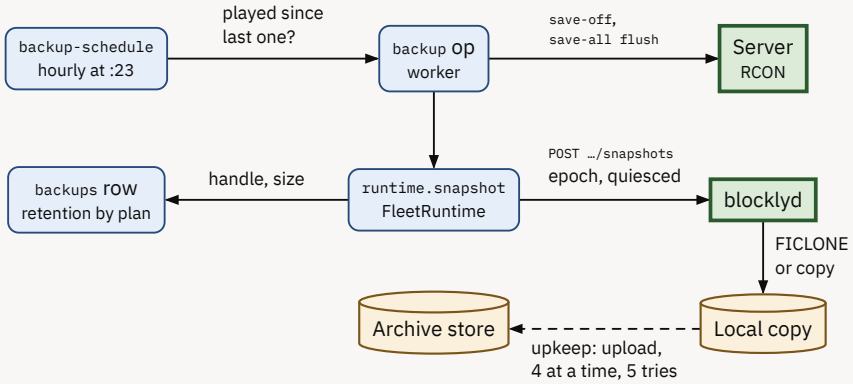


Figure 19: A scheduled backup on the fleet. `save-on` follows whatever happens. `apps/control/src/app/operations/handlers.ts:221` (capture), `apps/control/src/infra/fleet/fleet-runtime.ts:682` (snapshot), `fleet-runtime.ts:747` (upload).

1. **Schedule.** `backup-schedule` runs hourly at :23 (`apps/control/src/infra/pg/jobs.ts:287`) and queues a daily snapshot of every server played on since its last scheduled one; a server nobody played keeps its last backup. Plus also gets a weekly archive.
2. **Quiesce.** For a running server the worker pauses saving over RCON (`save-off`, `save-all flush`) and resumes it (`save-on`) afterwards, whatever happens. A stopped server saved as it stopped.
3. **Snapshot.** The runtime takes it. On the fleet, `fleet_archives` gets a row (`creating`), the node copies the data beside it (`local`), and the handle is issued at once: a restore on the same node can use the local copy immediately.
4. **Upload.** In the background, and by `runtime-upkeep` each minute until it succeeds: `POST …/snapshots/<id>/upload` to a presigned PUT, or in parts when it is larger than one carries, the size checked against the store's own (`uploading` → `ready`, `snapshot.uploaded`). A node packs a snapshot for one upload at a time: an export asked meanwhile waits its turn. Once a newer snapshot of the same server is uploaded, older local copies are dropped (`snapshot.local_trimmed`).
5. **Retention.** Plans keep `snapshotsKept` daily snapshots (Free 3, Plus 14); weekly archives (Plus) are kept 30 days; downloads are made on request (Free one a day, kept 7 days) (`apps/control/src/domain/account/entitlements.ts:143`).

CAVEAT

Until a snapshot's upload finishes, its only copy is on the node it describes. A node lost in that window loses that backup; a rebuild then uses the newest snapshot that did reach the store (`recover.older_copy`). Uploads are full archives every time; incremental ones are DESIGNED. A world whose archive is over one PUT's limit (5 GiB less 5 MiB) goes in parts: the control plane begins an S3 multipart upload sized for it and presigns a URL for each part, the runtime sends the parts and reports their ETags, and the control plane completes the upload, or aborts it when anything failed (`apps/control/src/infra/s3/s3-archive-store.ts:256`, `blocklyd/src/manager.rs:3038`). Only a fleet node without the `multipart-upload` feature still refuses such an archive before sending anything (`archive_too_large`, `blocklyd/src/manager.rs:3021`): its snapshots stay on its node, and it can't rest or move to another node.

21 The restore pipeline

Restoring replaces a stopped server's world with a backup's. IMPLEMENTED

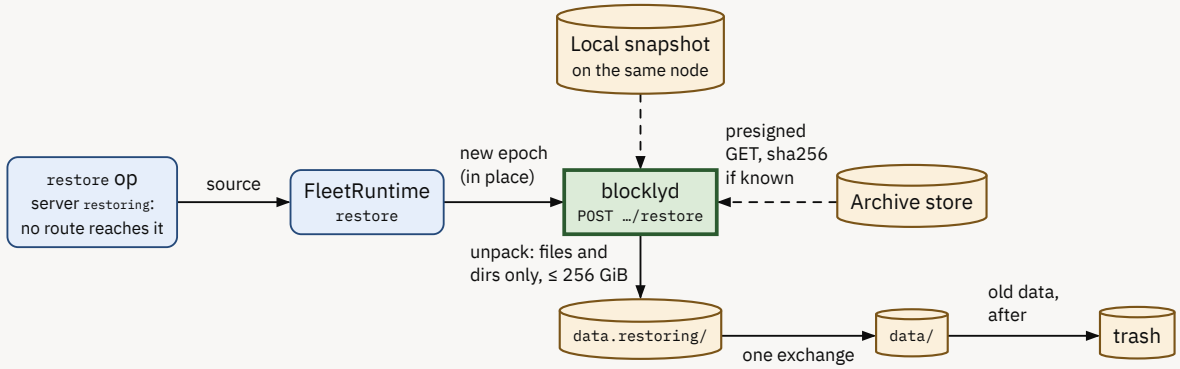


Figure 20: A restore. The new data is assembled beside the old, so a bad archive costs nothing. `blocklyd/src/manager.rs:1426`, `blocklyd/src/transfer.rs:161`, `apps/control/src/infra/fleet/fleet-runtime.ts:980`.

- **Sources.** A backup's snapshot handle when its runtime still holds it; else the archive store through a link made just before use (`handlers.ts:869`). On the fleet a snapshot is read from the node's local copy when the server is still on that node, else from the store, with its recorded sha256. An application archive (a download, the copy a world rests in, a move between runtimes) carries the one recorded when it was written; an uploaded world, and an archive made before migration 0044, carry none.
- **Unpacking is hostile-input safe.** Only regular files and directories; an absolute path or `..` refuses the whole archive; modes masked to `0755`; at most 2,000,000 entries and 256 GiB; files opened `O_NOFOLLOW`; links and devices skipped and counted. Restores from a local snapshot copy the tree and recreate symlinks as symlinks (they are never followed).
- **In place, a new epoch.** A fleet restore into a placed server takes a new epoch on the same node (`placement.restored`); a running copy is stopped first and started again after.
- **The application boots it while** `restoring`, where no route reaches it, so its access list is back on the files before anyone can join; a server that was stopped is stopped again.
- **The swap is one exchange.** `renameat2(RENAME_EXCHANGE)` puts the new data in place and the old where the new was, in one step, so a crash leaves the old data or the new, never neither. A marker in the new data tells which side of the swap a crash fell on, and blocklyd settles it when it starts and before each restore. A filesystem that can't exchange gets two renames, and the same recovery finishes the second (`blocklyd/src/store.rs:301`).

22 Relocation

Moving a fleet server to another node: an operator's move, a region change, a drain, or a start that needs room elsewhere. A move refused before anything was let go (no room anywhere, or a world too big to copy) leaves the server where it was, running if it ran; one that fails partway fails as any operation does (see the caveat below). IMPLEMENTED

The application takes a snapshot first, while the server still runs (`pre_relocate`, quiesced); then FleetRuntime moves it:

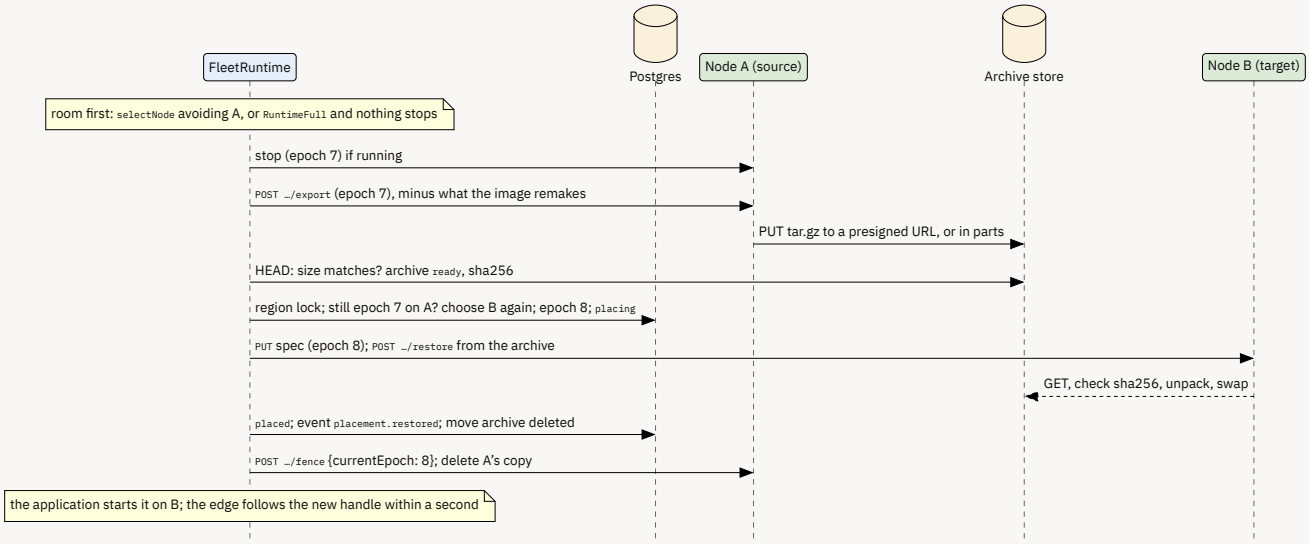


Figure 21: A planned move, epochs 7 → 8. `apps/control/src/infra/fleet/fleet-runtime.ts:1065` (relocate), `fleet-runtime.ts:2087` (export), `fleet-runtime.ts:2218` (finishMove).

Table 14: Relocation's failure behaviour.

If this fails	What happens
No node has room (before or after the export)	`RuntimeFull`. The export is discarded; a server that was running starts again on A. The server settles back where it was, running if it ran (`stayPut`, `apps/control/src/app/operations/handlers.ts:1092`); the relocation sweep asks again 15 minutes later, twice as long after each refusal in a row (up to a day), and only once there is room.
A world its node can't send (too big for one upload, on a node without parts)	Declined (`RuntimeUnsupported`), and recorded each time (`placement.move_declined`): an operator's request is dropped, a server that was running starts again on A, and it settles back the same way. Asked again, it is declined before anything stops (`fleet-runtime.ts:1947`).
The export or upload	Archive `failed`; the server stays on A at epoch 7, stopped if it was stopped for the move. The operation tries once more, then fails for good (below).
The placement changed meanwhile	`StaleHandle`; the copy is discarded (`move.export_superseded`).
Crash after epoch 8 was chosen	The row stays `placing` with `completing = move`; the next start or relocation finishes it from the archive (`#finishMove` is idempotent).
Restore on B	The row stays `placing`; retrying restores again. A is not fenced until B holds the world.
The fence or delete on A	Logged; the heartbeat answer and `tidy()` fence and remove it later.

- Only what the image can't remake is moved: the server jar and unpacked libraries are excluded (`reconstructible`) and remade at first start, so a move costs an export and a restore of the world itself: seconds for a small world. Measured: a 1 GB world exports in 14.3 s and restores in 10.8 s on the test VM (§F).
- **A start that needs room** (`#startElsewhere`) uses the same steps, then claims room on B and starts it there (`placement.moved_for_room`).
- **Between runtimes** (fleet to Fly, Fly to Boat …) a server moves through a backup any runtime can restore, only when an operator asks (`scripts/runtimes.ts move`), at its next quiet moment (§5).

CAVEAT

A move that fails partway stops the server. An export, upload or restore that fails is tried once more (`apps/control/src/app/servers/transitions.ts:24`); when that fails too, the runner lets the operation settle what it owns, which stops a server still reading `relocating` (`stopWhatFailed`, `apps/control/src/app/operations/handlers.ts:176`), and records it `failed` (`apps/control/src/app/operations/runner.ts:165`). Its world is whole on A, or placing on B; the sweep leaves a move the platform made that failed to its owner, whose Try again runs the same move (`apps/control/src/app/operations/schedules.ts:341`). Only refusals settle back on their own.

23 Draining

`scripts/fleet.ts drain <node>` sets the node's lifecycle to `draining`: no new placements go there, and its servers read as misplaced (`isPlaced` is false, `fleet-runtime.ts:1295`). The `relocations` sweep then moves them each minute, three at a time, each once another node in its region has room for it: a stopped server at once, a running one only while nobody is

on it. A move refused or failed lately waits 15 minutes before the sweep asks again, twice as long after each refusal in a row, up to a day; a world too big to copy never leaves, and says so each time. `undrain` stops the drain. An operator's `move <server>` doesn't wait: it starts the relocation at once, whoever is playing. IMPLEMENTED

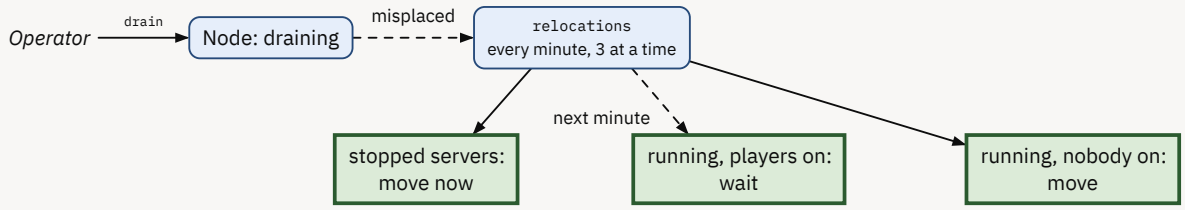


Figure 22: A drain never stops a busy server. `apps/control/src/app/operations/schedules.ts:326`.

WHY

Moving a running server costs its players a reconnect; waiting costs nothing. A drain is for planned work (an upgrade, a decommission), where minutes don't matter. An operator who can't wait moves servers explicitly (`move <server>`).

24 Node loss

What happens when a host dies, from the first missed heartbeat to its servers running elsewhere. IMPLEMENTED, except the parts marked.

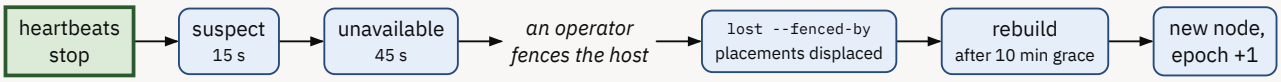


Figure 23: Node loss. Nothing to the right of "unavailable" happens without an operator.

1. **Detection.** Suspect after 15 s of silence, unavailable after 45 s. No new placements; its servers read `unknown`. Players connected to it are already disconnected if the host is dead, or still playing if only the network to the control plane is cut (§25).
2. **Fence, physically.** The operator makes sure the host is stopped and stays stopped: power-off in the provider's console, rescue boot, or network removed. No provider API is wired to do this (DEFERRED).
3. **Declare.** `scripts/fleet.ts lost <node> --fenced-by <how> --reason <why>`. Refused while the node still beats (`node_is_beating`) unless forced. Its placements become `displaced`, its observations are deleted, events `node.lost` and `placement.displaced`.
4. **Grace, then rebuild.** The application marks each server `host lost` when it first sees it, and the relocation sweep rebuilds it after `HOST_LOSS_GRACE_MS` (10 minutes), asking once more whether the host answers (`apps/control/src/app/operations/schedules.ts:82`). The rebuild places it on another node in its region under a new epoch and restores its newest **uploaded** snapshot (`#rebuild`, `fleet-runtime.ts:2165`).
5. **Route.** The new handle names the new node; the edge follows within a second.

Table 15: Node loss, in answers.

Question	Answer
What is lost?	Play since the newest snapshot that reached the archive store: up to about 24 hours (scheduled snapshots are daily, for servers played since the last one), plus any upload still pending.
How long are players out?	Detection (45 s) + the operator's response + 10 min grace + the restore (seconds per GB).
With one node?	The servers wait for the host. Declared lost with no other node, they wait too: the sweep asks for a rebuild only once a node in the region has room, and then rebuilds them by itself.
If the host comes back?	Its heartbeat is answered with lease 0 and fences for every rebuilt server's copy; nothing there runs again. Every copy whose epoch ended `lost` is reported as a fork (`fork.detected`), whether or not it holds newer play: nothing compares the data. `tidy()` never deletes such a copy: its owner decides.
If the operator was wrong?	`reinstate` puts the node back to `active`; certificates stay pinned through `lost`.

25 Split brain

The one failure the fleet does not prevent on its own: a node cut off from the control plane but not from players. IMPLEMENTED as described; automatic prevention DEFERRED.

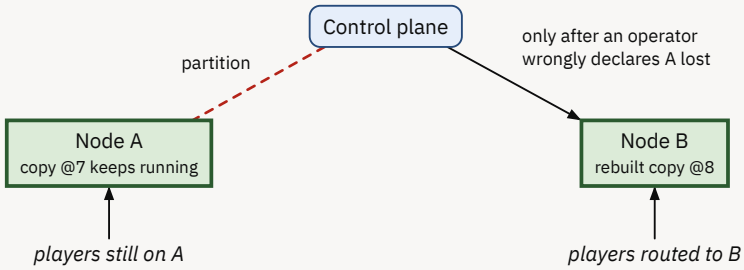


Figure 24: Two copies run only if an operator declares a reachable-by-players node lost. The world forks.

What stops it, and what does not:

- **The lease** stops A restarting or resuming anything unasked after 120 s without an answer. It does not stop A's running copy: the lease never stops anything (§15).
- **Epochs** make A's copy unusable to the control plane: any request at epoch 7 is refused once A has heard of epoch 8. But A can't hear while partitioned.
- **The operator's physical fence** is the guard: `lost` requires saying how the host was stopped, and is refused while the node beats. An operator who declares a partitioned-but-alive node lost gets two copies.
- **Stale connections.** mc-router doesn't close a connection when its route changes, so players on A's copy keep playing on it until A hears its fence and stops it, as the two-node experiment showed.

When A returns, its first heartbeat is answered with fences; its copy stops and never runs again, and the fork is reported (`fork.detected`). Nothing is merged automatically.

> **WHY**
> **Why not automatic failover.** It needs a fence that doesn't depend on the node: a provider power-off that stays off, or a node that stops its own servers when its lease runs out. The second turns every control-plane outage longer than the lease into a fleet-wide outage; the first isn't built. So the operator's confirmation is the fence (`docs/fleet.md:255`). Closing stale connections at the edge (a relay in the edge agent) is DESIGNED, not built (`docs/fleet.md:426`).

PART V

Network and providers

How players reach a server, which providers sit under which runtime, what Boat is today, what running machines directly means, how to choose between them, and the canary that compares them.

26 Edge routing

Players reach every server through one edge: `itzg/mc-router` 1.47.1 and a small agent (TypeScript, compiled with Bun) in one image, the only public Minecraft address. The agent is the only code that knows mc-router exists (`apps/edge/agent.ts:1`). IMPLEMENTED

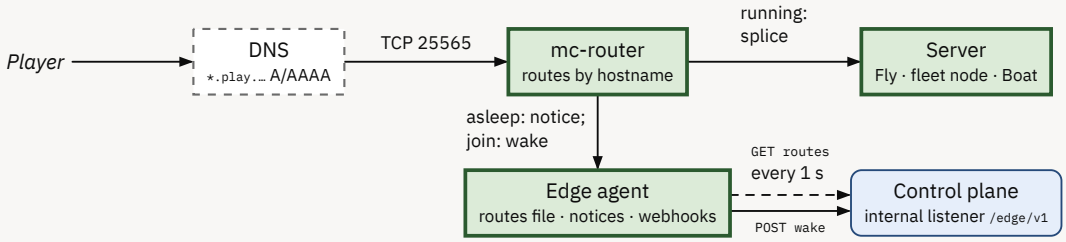


Figure 25: The player's path. Routes are pulled, never pushed: every edge converges and a restart loses nothing. `apps/edge/agent.ts:44`.

Addressing. A server is `<slug>.<PLAY_DOMAIN>` (port omitted at 25565). Nothing hostname-shaped is stored: addresses are the slug plus configuration, so a domain move is a config change with the old domain kept as an alias until joins to it stop (`play_domain_joins`). SRV records are not used: they rewrite the handshake hostname, which is what the edge routes on.

Routes. `GET /edge/v1/routes` answers every server with a handle whose status is routable, with its play hostnames, a destination (`host:port`, from `endpoint(handle, 'game', 'edge')`) and an optional state (`asleep`, `restarting`). The answer is computed from the database on every request and carries an ETag, so most polls are a 304 (`apps/control/src/app/edge/service.ts:64`).

Table 16: Routes by status. `apps/control/src/app/edge/service.ts:75`, `apps/control/src/domain/server/lifecycle.ts:264`.

Server status	Route	Destination
running	yes, no state	The runtime's edge endpoint.
stopped, stored, starting	yes, `asleep`	The endpoint, or `0.0.0.0:0` on runtimes whose address isn't stable (Boat): "nothing listens on port 0, so a status ping fails at once and gets the asleep message, and a join asks for a wake".
updating, or a restart, restore or move of a running server	yes, `restarting`	As above.
provisioning, restoring, storing, failed, deleted	no route	Unrouted on purpose: a world being restored gets its access list back before anyone can join.

- **Destinations per runtime**: Fly `<app>.flycast:25565` on the org's private network (the server's app has no public address); fleet `<node edge address>:<host port>`; Boat the sandbox's public IPv6 while it runs; Docker the container on the games network.
- **Failure behaviour.** If the control plane is unreachable the agent keeps the last routes; a hostname newer than the last poll answers "no route yet" (players see "starting"); a server the database says runs but that doesn't answer gets the restarting notice, re-probed every 2 s.
- **Stale connections.** mc-router doesn't close a connection when its route changes. A planned move stops the source first, so it can't matter there; after a wrongly declared loss it would (§25). A relay in the agent that closes connections whose destination left the route set is DESIGNED (`docs/fleet.md:426`).

CAVEAT

The fleet's network path is not built. The edge and control plane run on Fly's private network; fleet nodes would need to be reachable on theirs ("a private network is required between the control plane, the edge and every node", `docs/fleet.md:403`). Nothing in the repository creates that link, and reaching nodes from Fly is an open question. The control plane's node endpoint listens on `[::]:8443` by default, IPv6 and IPv4, as Fly's private network needs (`apps/control/src/config/load.ts:77`), but no environment runs a node yet.

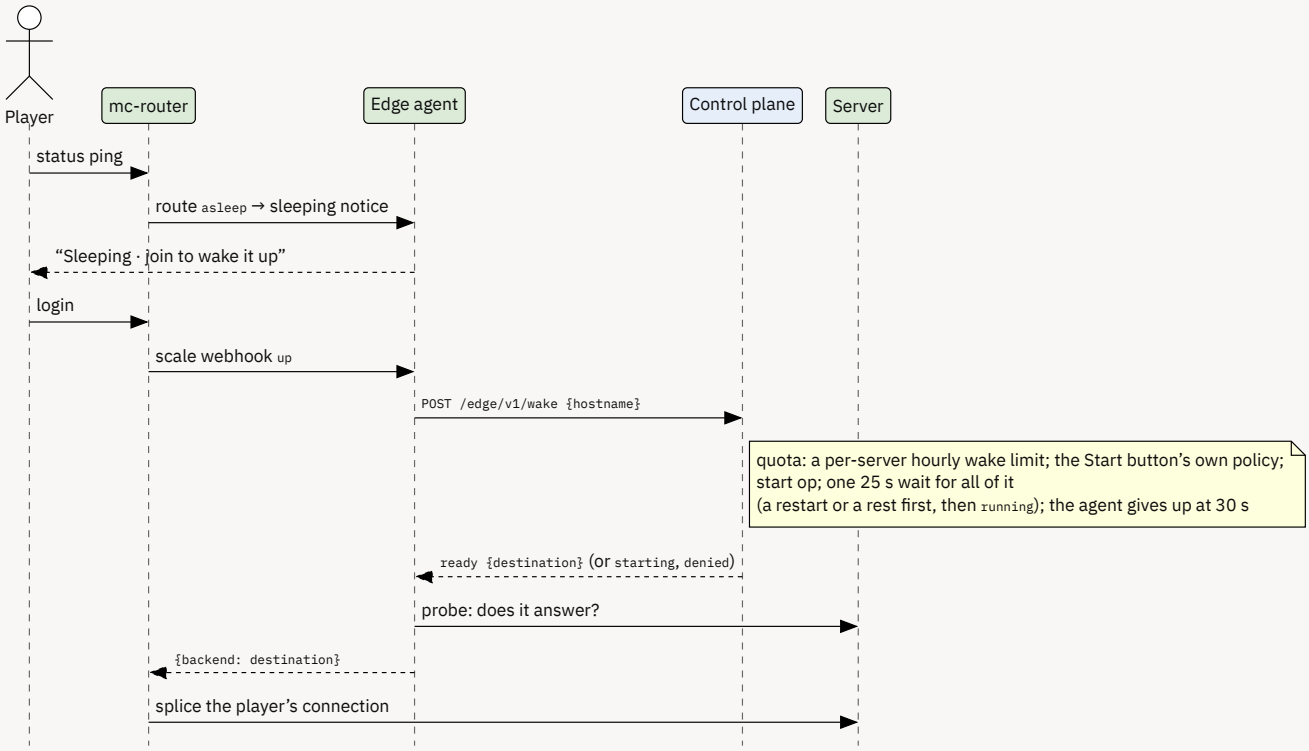


Figure 26: Wake on join. A run started by a connection is on probation: if nobody completes a login within 5 minutes it stops (`nobody_joined`). `apps/control/src/app/edge/service.ts:93`, `apps/control/src/app/operations/schedules.ts:123`.

27 Provider layering

Three ways of running a server, and the providers under each. IMPLEMENTED for the runtimes; direct-host providers RESEARCH ONLY.

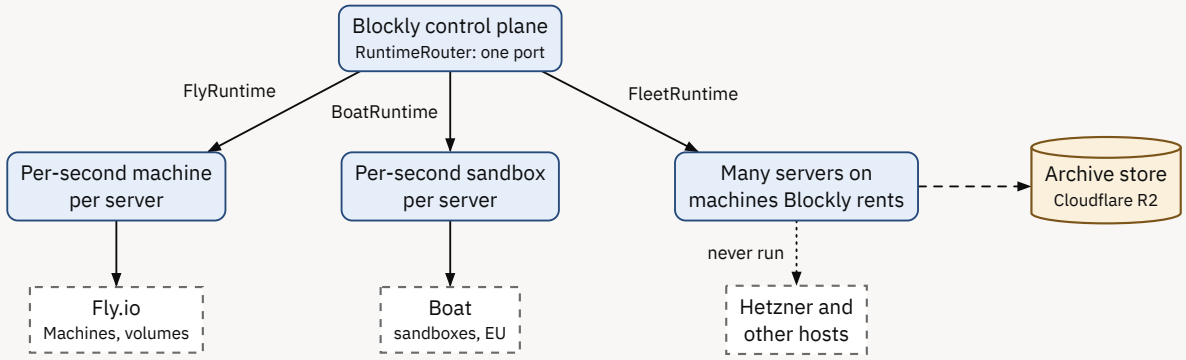


Figure 27: Provider layering: a machine per server (Fly), a sandbox per server (Boat), or many servers on each machine (the fleet).

Table 17: The three runtimes behind the port (Docker is local development).

	Fly	Boat	Fleet
What a server gets	Its own Fly app on its own private network, one volume, one Machine (performance CPUs), a Flycast address	A whole Linux VM (“sandbox”) running one container; billed per second while it runs, free stopped	A hardened container on a rented machine Blockly runs blocklyd on
Snapshots	Fly volume snapshots (not point-in-time: writes 4 s after the ask were held)	A tar.gz in a snapshot volume inside the sandbox	Local copy (FICLONE or copy), then upload
Limit	Org machine limit: `floor((limit - platform) / 2)` servers	Daily start quota by plan; EU only	Nodes you add
Configured	Production and staging	Nowhere	Nowhere

- **Other providers**: Cloudflare (DNS, R2 archive store, ACME DNS-01, the web app’s Worker), Polar (billing, merchant of record), an SMTP provider (unnamed), Modrinth and Mojang (catalog, profiles).
- **Archives are the only thing runtimes share.** Snapshots never cross runtimes; a move between runtimes goes through an archive any runtime can restore, so nothing of one provider is ever handed to another (`apps/control/src/app/operations/handlers.ts:1110`).

28 Boat, as it actually is

Boat runs each server in its own sandbox: a Linux VM, billed by the second while it runs and free while stopped. IMPLEMENTED, configured in no environment, and not used by any server.

Table 18: Boat's status.

Question	Answer, from the code and its report
Where?	Its adapter is `apps/control/src/infra/boat`.
Enabled?	No. Not the default (`RUNTIME_PROVIDER` defaults to `docker`; environments set `fly`), and no environment lists it in `RUNTIME_PROVIDERS`.
Verified live	Create → running; a new address on every resume; a Fly machine reached the sandbox's IPv6 through mc-router; firewall reopened per start; recovery from a crash, an in-VM reboot and a Boat-initiated stop; backup and restore in the sandbox; a parked sandbox left alone and stopped by Boat.
Not verified	Export to an archive store and restore onto a new sandbox (unit tests only); a resize; a real player joining through the edge; the start-headroom alert.

CAVEAT

On Boat the game port is public: published on all interfaces, opened in the sandbox's firewall, reachable over IPv6 without the edge (no edge rate limit, no session hints), and Boat itself exposes ports 22 and 8911 (`apps/control/src/infra/boat/sandbox-scripts.ts:70`). It is the one exception to "no public address on a game server" (`docs/architecture.md:2320`).

29 Direct-host providers

The fleet runs on machines Blockly rents directly. Provisioning hosts from provider APIs is DEFERRED: a host arrives with Docker, the binary, its config, the CA certificate and a token, by cloud-init, an install image or by hand.

- **Trust comes from the one-time token**, spent before any workload runs, not from a provider's instance identity.
- **Worlds stay on local disks**, not on provider block volumes.
- **Automatic failover** would need a power-off that stays off; today the operator fences (§25).

30 Choosing a runtime

What decides which runtime suits a server. RESEARCH ONLY

- **A runtime billed per running second** (Fly, Boat) keeps only a sleeping server's disk or snapshot, never its compute.
- **Machines are paid for running or not.** On the fleet what matters is running utilization: the GB-hours servers run, divided by the GB-hours the nodes offer. Running admission, which holds memory only for running servers (§8), is what makes it high.
- **Boat's limits** are a daily start quota by plan and Europe only.
- **Owning machines suits** always-on servers, regions Boat doesn't serve, and servers where single-thread speed matters.

31 The canary

A canary is not a flag. It is a placement rule (or explicit moves) that sends some **new** servers to a runtime other than the default (`docs/runtimes.md:3`). IMPLEMENTED, with no rules defined: no deployment runs a canary.

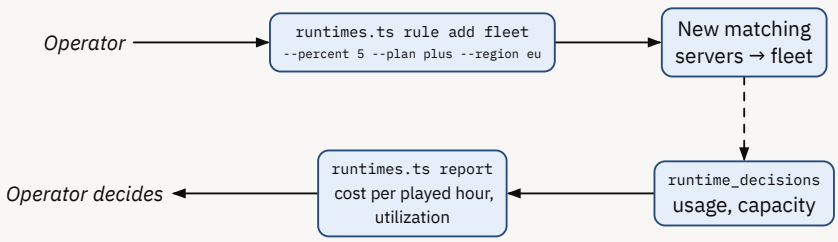


Figure 28: The canary loop. Turning a rule off affects only new servers; servers already on the canary stay until moved one by one.

- **Choosing servers**: an allowlist of accounts, regions, plans, and a stable percentage (hash of rule and server). Or explicit moves: `bun scripts/runtimes.ts move <id>… --to fleet`.
- **Measuring**: `bun scripts/runtimes.ts report` gives per runtime the servers, running and played hours, GB-hours, node capacity, idle memory and running utilization. Bandwidth and per-server CPU are not measured (printed as blind spots) (`apps/control/src/app/runtimes/economics.ts:8`).

- **Stopping**: `rule off <rule>` or percent 0. There is no rule delete. Removing a runtime from `RUNTIME_PROVIDERS` is not a stop: its servers become foreign.

PART VI

Security and operations

What is protected and from whom, what operators can see and do, where everything is deployed, and what happens when each part fails.

32 Security model

Who can do what, at each boundary, and how far isolation really goes. IMPLEMENTED unless marked.

Table 19: Authentication at each boundary.

Boundary	Authentication
Browser → web → control API	Better Auth: email and password (10 characters minimum, verification email, 1-hour reset links), optional Google and GitHub; session cookie host-only on the web origin. tRPC mutations refuse cross-site requests and untrusted origins. Admins are rows in `platform_admins`; non-admins get `NOT_FOUND`. No 2FA or passkeys.
Browser → realtime	`Origin` must be a web origin, plus a 60-second ticket (JWT) for this deployment.
Edge → control (internal listener)	Shared `EDGE_TOKEN`, compared in constant time; reachable only on the private network.
Operators → control (internal listener)	One shared `OPERATOR_TOKEN`; the operator's name is the self-declared `x-operator` header. Off unless set; set in no environment.
Game runtimes → artifacts	A per-server token derived from the runtime key, plus a hash the file must match.
Polar → billing webhook	Polar's signature (`whsec_…`).
Control ↔ fleet nodes	Mutual TLS 1.3 under a per-deployment CA: one-time enrollment token, then pinned node certificates (30 days); the control plane's client certificate lasts 2 days, renewed daily (§12).
Public → edge	None by design: anyone may connect. Wakes are bounded by a per-server hourly limit, set in private configuration, and a 5-minute probation; connections are limited per server and per address at the edge, ahead of mc-router's global limit.

Isolation, as it is (no more than the code does):

Table 20: Isolation per runtime. `blocklyd/src/runtime/docker.rs:138`, `apps/control/src/infra/fly/fly-runtime.ts:930`.

Runtime	What separates servers	What doesn't
Fly	Each server is its own Fly app on its own private network; no public address; reached only through Flycast from the org network; RCON only there. Fly's VM isolation.	Relies on Fly's documented behaviour; the edge and control plane share the org network.
Fleet	A container per server: a numeric user, never root (blocklyd refuses to start with uid or gid 0 for workloads or their data), every capability dropped, `no-new-privileges`, read-only root filesystem (host default), private IPC, PID limit, hard memory limit without swap, logs bounded, a bridge network with inter-container traffic off, ports only on configured addresses. A request cannot relax host policy.	**Containers share the host's kernel**: a kernel exploit from a plugin or mod escapes. No seccomp or AppArmor profile beyond Docker's defaults; no user namespaces; egress open; cloud metadata and host addresses not blocked; no CPU quota; no disk quota.
Boat	A VM per server.	The game port is public over IPv6, bypassing the edge (§28).
Docker (local)	Containers on a network only the edge joins.	Development only.

- **Reading worlds safely.** blocklyd reads a world as root, so every walk opens each directory relative to its parent with `O_NOFOLLOW`, never follows a link, skips entries swapped mid-walk, never opens a FIFO or device, and cuts a file that changes to the size its header states; a race test swaps continuously while walking (`blocklyd/src/tree.rs:92`).
- **A node's own surfaces.** Its API asks every caller for a certificate under the fleet CA. Its ops listener (health, metrics naming every workload) asks for none, so blocklyd refuses to start with it bound anywhere but loopback or a private address (`blocklyd/src/config.rs:513`). An enrollment answer must name the CA the node was provisioned with, or the node writes no identity (`blocklyd/src/fleet/enroll.rs:94`).
- **Secrets.** Per-server secrets (RCON passwords, artifact tokens) are derived from a versioned key, never stored. On Fly they are app secrets, never in a machine's config. On a node they exist only in Docker's container config, never in records, logs or answers.

33 Observability

There is no metrics pipeline in the control plane; answers are durable rows read with SQL. IMPLEMENTED

Table 21: What can be seen.

Signal	What and where
Health	`GET /api/health` (Fly checks it every 15 s); TCP checks on edge and realtime; blocklyd `/healthz` and `/readyz` (Docker up and reconciled).
Operations	`server_operations`: kind, status, step, the owner's error words and the operator's detail, per operation.
Audit	`audit_log`: every privileged action and platform decision (`server.runtime_moved`, `server.rebuilt_from_backup`, `platform.controls_changed`, …).
Runs and play	`power_intervals` (with runtime and `woken`), `server_usage_days`, presence tables.
Placement	`runtime_decisions`: every placement, fallback and move with what was considered.
Fleet	`fleet_events` (47 kinds, kept 90 days; §E); `/fleet/v1/summary`; node views with memory running, placed and observed.
Live hints	The `blockly_events` NOTIFY channel → realtime → browsers.
Alerts	Admin alerts (`blocked_operations`, `purge_overdue`, `catalog_stale`, `worlds_outgrow_plan`, `extra_play_unsent`, `extra_play_waiting`, `billing_order_unknown`): live in admin and emailed once. No fleet alerts: the watch list in `docs/fleet-operations.md` is DESIGNED.
Nodes	blocklyd `/metrics` (OpenMetrics, 24 families, loopback by default): operations, HTTP, auth failures, reconciles, workloads by state, per-workload memory, CPU and disk, heartbeats, contact age, lease left, renewals, snapshot bytes.
Logs	Control plane: plain lines to stdout. Game logs through each runtime's log source (Fly NATS, Docker, blocklyd NDJSON, Boat exec).

WHY

A young product has more questions than dashboards; durable columns answer any question later, with SQL, without deciding up front what to count. The cost is that nothing pages anyone: the fleet's alerts are a list in a runbook, not code.

34 Operator workflows

Everything an operator does goes through a script or the admin UI. IMPLEMENTED

Table 22: Operator entry points. `scripts/fleet.ts:1`, `scripts/runtimes.ts:1`, `scripts/staging.ts:1`.

Command	Does
`bun scripts/staging.ts up | start | stop | status | env`	Make and deploy staging, start it, stop every machine (and say so), show it, write cloud credentials. `down` destroys everything, and is only for retiring staging.
`bun scripts/staging-check.ts [--keep]`	One Free server through its whole life on staging, timed.
`bun run smoke`	The product end to end on the local stack.
`bun scripts/runtimes.ts …`	`summary`, `rules`, `rule add|set|on|off`, `where <server>`, `move <server>… --to <runtime>`, `stay`, `report`.
`bun scripts/fleet.ts …`	`ca`, `token <region> | --node <node>`, `nodes`, `node`, `label`, `drain`, `undrain`, `lost --fenced-by --reason [--force]`, `reinstate`, `retire`, `clear-quarantine`, `placements`, `placement`, `move [--to]`, `events`, `summary`.
`bun scripts/architecture.ts`	Renders this document.
Admin UI	Accounts (suspend, plan, limits, admins), servers across providers, platform kill switches and caps, stuck operations (retry, discard), audit log, reports, trusted mods, curated packs.

Fleet runbooks live in `docs/fleet-operations.md`: a new deployment, adding a node, day to day, a node that stops answering, declaring a node lost, a lost node coming back, retiring, certificates, upgrading blocklyd (replace the binary and restart: servers keep running), disk, two hosts with one identity.

A restored database keeps the copies it doesn't know. A copy whose server the database doesn't know is fenced, kept and reported (`copy.orphaned`), and the hourly `orphans` sweep logs it each time it sees it, for an operator, and never destroys it: after Postgres is restored from a backup, that is every server made since, and a destroyed world can't be brought back (`apps/control/src/app/operations/schedules.ts:808`). Only a copy whose server was purged is destroyed, into the node's `trash/` for 24 hours.

35 Deployment topology and trust boundaries

As configured in the repository: production in Terraform, staging by script. IMPLEMENTED as configuration; production's application status unknown.

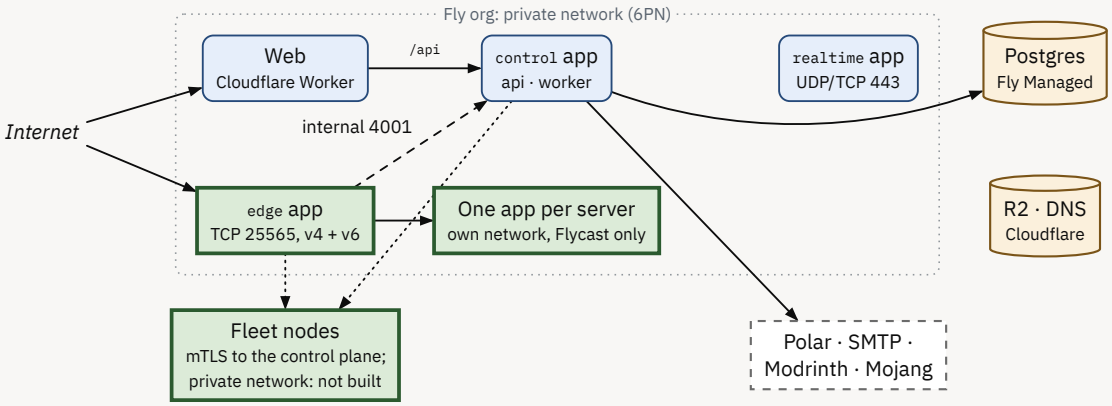


Figure 29: Production as configured (`infra/terraform`). Staging differs: a Postgres machine, Tigris, nip.io play addresses. Dotted: designed, not built.

Table 23: Where secrets live. `infra/terraform/modules/control/main.tf:63`.

Holder	Secrets (names only)
`control` and `realtime` apps	Every control-plane secret: database URLs, `AUTH_SECRET`, OAuth clients, `WEB_PROXY_SECRET`, `REALTIME_TICKET_SECRET`, `EDGE_TOKEN`, `RUNTIME_SECRETS_KEY`, `FLY_API_TOKEN`, `SMTP_URL`, archive keys, Polar token and webhook secret, `CLOUDFLARE_DNS_API_TOKEN`; and when used, `BOAT_API_TOKEN`, `FLEET_CA_KEY`, `OPERATOR_TOKEN`.
`edge` app	`CONTROL_URL`, `EDGE_TOKEN`.
Web (Cloudflare Worker)	`WEB_PROXY_SECRET`, a secret of each version.
A server's Fly app	Its own per-server secrets, as app secrets.
A fleet node	The fleet CA's certificate (public) and a one-time token; never the operator token.

- **Roles, not services.** One control-plane image in three roles; `api` and `worker` are process groups of one Fly app (migrations run as its `release_command` before any machine takes the image); `realtime` is its own app because public ports belong to an app and both HTTPS and the WebSocket fallback need TCP 443.
- **One Fly org per environment**, because Fly's private network and billing are per org.
- **Game servers reach the control plane only through the public artifact endpoint**: their app networks have no route to the org network.
- **No deploy pipeline.** Deploys are `fly deploy` and Terraform by hand, the staging script, and wrangler for the website's Worker.

36 Failure matrix

What fails, what players see, and what recovers it. "Auto" means without a person.

Table 24: Failures and recoveries. `docs/fleet.md:387`, `apps/control/src/app/operations/runner.ts:86`.

What fails	Effect	Recovery
A game server crashes	Players disconnected.	Auto: Fly restarts it (on-failure ×3); a fleet node restarts it under its lease (3 tries, backoff), or reports why it didn't (`insufficient_capacity`); reconcile marks it `stopped{crash}` after that, with an out-of-memory kill noted.
A worker dies mid-operation	The operation stalls.	Auto: pg-boss retries (per kind); operations are convergent; per-server FIFO keeps order. A blocked key is retried or discarded by an admin.
Control plane down	No starts, stops or changes; running servers keep running; the edge keeps its last routes; nodes restart nothing unasked after 120 s.	Auto when it returns: reconcile, heartbeats resync everything.
Postgres down	The control plane can't act.	Restore from the provider's backups; a restored database adopts newer node epochs and fences copies it doesn't know (§14), which the hourly sweep keeps and logs for an operator (§34).
Edge down	Nobody can join; running sessions through it drop.	Fly restarts the machine; production runs two.
blocklyd crashes	Nothing: servers aren't its children.	Auto: it reconciles from disk and Docker in about a second; nothing that died meanwhile is restarted.
Docker restarts on a node	With `live-restore`, nothing; without, servers stop.	Reported, not restarted.
A node reboots	Its servers stop.	Auto: resumed once a lease is held, fences first.
A node is silent	No new placements; its servers read `unknown`.	Operator (§24).

What fails	Effect	Recovery
A node is gone for good	Its servers are down.	Operator fences and declares it lost; auto rebuild after 10 min from the newest uploaded snapshot; play since then is lost.
A partition hides a live node	Its players keep playing.	Don’t declare it lost (§25).
Disk full on a node	Creates, snapshots, exports and restores refused (`507`), a URL restore also when its download wouldn’t fit; running servers untouched.	Operator frees space; placement avoids the node.
Archive store down	Snapshots stay local; moves, rebuilds, rests and wakes from rest wait.	Auto: uploads retry each minute (5 tries per snapshot); a rest that failed is tried again a day later.
A provider is full	`RuntimeFull`: a start goes back to sleep; a new server a rule placed falls back once to the default; a move or rebuild is refused before anything stops, and the server stays as it was.	Auto on the next join or press; the relocation sweep asks for a move again once there is room.
Boat out of starts	Wakes refused until the window resets.	Auto, later.

PART VII

Status

What exists, and what does not.

37 Status

Every capability, its status, and where it lives.

Table 25: Status of everything this document describes.

Capability	Status	Evidence and notes
Product and control plane		
Servers, worlds, settings, revisions, rollback	IMPLEMENTED	`apps/control/src/domain`, `app/servers`, `app/revisions`.
Operations queue, per-server FIFO, retries	IMPLEMENTED	`infra/pg/jobs.ts`, `app/operations/runner.ts`.
Sleep on idle, wake on join, wake probation and quota	IMPLEMENTED	`schedules.ts:649`, `app/edge/service.ts:93`.
Resting worlds (store, unstore)	IMPLEMENTED	Free after 14 idle days, Plus after 30.
Backups: daily snapshots, weekly archives, downloads	IMPLEMENTED	`backup-schedule`, `handlers.ts:737`.
Plans, included hours, Polar billing	IMPLEMENTED	`entitlements.ts`, `app/billing`.
Paid extra hours	DEFERRED	`mayBuyMore` false on every plan until metering is end to end.
Edge: routing, notices, wake	IMPLEMENTED	`apps/edge`.
Edge relay closing stale connections	DESIGNED	`docs/fleet.md:426`.
Runtimes		
The runtime port	IMPLEMENTED	`app/ports/runtime.ts`.
Fly runtime	IMPLEMENTED	The runtime staging runs and production configures.
Boat runtime	IMPLEMENTED	Configured nowhere.
Docker runtime	IMPLEMENTED	Local development.
Several runtimes in one deployment	IMPLEMENTED	Configured nowhere.
Placement rules (the canary), overflow, first-start fallback	IMPLEMENTED	No rules exist.
Moves between runtimes through archives	IMPLEMENTED	Operator-only.
Archives larger than one PUT, in parts	IMPLEMENTED	Every runtime; tested against RustFS, never against R2.
Canary report	IMPLEMENTED	Bandwidth and per-server CPU unmeasured.
Moving a waking server to the cloud when the fleet is full	DESIGNED	The “hybrid”; not built.
Cost-aware placement, rebalancing	DEFERRED	Not built.
Fleet		
blocklyd 0.2: protocol, hardening, epochs, lease, resume, renewal	IMPLEMENTED	142 Rust tests, and 8 more against a real Docker.
Enrollment, heartbeats, health, quarantine	IMPLEMENTED	`registry.ts`, `node-endpoint.ts`.
Placement policies, running admission, moves for room	IMPLEMENTED	`placement.ts`.
Snapshots (local then uploaded), moves, rebuilds, drain	IMPLEMENTED	One-node e2e in CI; two-node e2e local only.
Fleet in any environment	CANARY ONLY	Never configured; never run on independent machines.
Private network from Fly to nodes	RESEARCH ONLY	Open spike; nothing built.
Host provisioning from provider APIs	DEFERRED	Hosts are provisioned by hand or cloud-init.
Provider power fencing; automatic failover	DEFERRED	The operator is the fence (§25).
Fleet alerts	DESIGNED	A watch list in the runbook only.
Reflink snapshots on a real reflink disk	IMPLEMENTED	Never measured: the test kernel had ext4 only.
ZFS	RESEARCH ONLY	Not adopted.
gVisor, Kata or Firecracker isolation	DESIGNED	`docs/fleet.md:453`.
Disk quotas per world	DEFERRED	Measured, not enforced.

38 Current limitations

What is true today and matters before relying on the parts it touches. Each is either stated in the code’s own documents or found while writing this one.

Product-wide

- Nothing pages anyone: there is no metrics pipeline in the control plane and no fleet alerting.

Runtimes

- Fly never starts a server on another host: one whose host is full waits for room there.
- Boat: the edge waits 25 s, Boat wakes in about 34 s; game ports are public.

Fleet and blocklyd

- Never run on independent machines; the network path from Fly to nodes is not built.
- Split brain is prevented only by the operator's physical fence; stale connections survive a route change.
- No CPU ceiling for fleet servers; density never measured; the host reserve is a default, not a measurement.
- Snapshot uploads are full archives every time. One larger than a PUT carries (5 GiB less 5 MiB) goes in parts, sent one after another; an upload in parts cut short by a crash begins again rather than resuming, and the store drops the parts left behind (R2 after 7 days).
- A move that fails partway (an export, upload or restore error, not a refusal) leaves the server stopped and `failed` until its owner tries again.
- Node-side certificate renewal, resume after a real reboot, and reflink snapshots have no test on a real host (renewal has one through the real heartbeat loop, in process).

Appendices

Reference material: interfaces, entities, protocol, configuration, signals, measurements, tests and words.

A Interfaces

A.1 MinecraftRuntime

The port every runtime implements (`apps/control/src/app/ports/runtime.ts:192`). Handles are branded opaque strings.

```
interface MinecraftRuntime {
  readonly provider: string                    // 'fly' | 'boat' | 'fleet' | 'docker' | 'fake'
  readonly serverCeiling: number | null        // most servers it can hold; null: no ceiling
  owns(h: RuntimeHandle): boolean
  ownsSnapshot(s: SnapshotHandle): boolean
  hasRoom(p: Placement, size: { memoryMb; storageGb }): Promise<boolean> // a hint
  // idempotent and convergent; leaves the server started
  ensureProvisioned(key, p: Placement, spec: RuntimeSpec, progress: ProgressSink,
                    install?: InstallSeed | null): Promise<RuntimeHandle>
  apply(h, spec): Promise<RuntimeHandle>       // restarts if running
  start(h): Promise<RuntimeHandle>             // may return a new handle (Boat)
  restart(h): Promise<void>                    // same compute, same handle
  stop(h): Promise<void>
  forceStop(h): Promise<void>
  waitRunning(h, signal: AbortSignal): Promise<void>
  readonly snapshotLifetimeDays: number | null
  snapshot(h): Promise<{ snapshot: SnapshotHandle; sizeBytes: number; at: Date }>
  goneSnapshots(ss): Promise<ReadonlySet<SnapshotHandle>>
  deleteSnapshot(s): Promise<void>             // may throw RuntimeUnsupported
  restore(h, from: RestoreSource, spec, progress): Promise<RuntimeHandle>
  relocate(h, to: Placement, spec, progress, from?: SnapshotHandle): Promise<RuntimeHandle>
  exportSnapshot(s, target: ArchiveTarget): Promise<{ sizeBytes: number; sha256: string }>
  exec(h, command: readonly string[], timeoutSeconds: number): Promise<ExecResult>
  decommission(h): Promise<void>               // compute gone, storage kept
  release(h): Promise<RuntimeHandle>           // compute and storage gone; world is elsewhere
  adopt(key, p, spec): Promise<RuntimeHandle>  // a handle for a world arriving
  destroy(target: RuntimeKey | RuntimeHandle): Promise<void>
  observe(h): Promise<RuntimeObservation>      // state, exit, hostLost?, handle?
  observeChanged(since: Date): AsyncIterable<{ key; handle; observation }>
  isPlaced(h, p): boolean
  sameCompute(a, b): boolean
  inventory(): AsyncIterable<{ key; handle }>
  endpoint(h, port: string, audience: 'edge' | 'control'): Endpoint
  readonly stableEndpoints: boolean
  tag(servers: ReadonlyMap<RuntimeKey, RuntimeTags>): Promise<number>
  locate(h): RuntimeLocation
  prices(h, size): RuntimePrices | null        // list prices, US cents
  capacity(): Promise<RuntimeCapacity | null>  // machines paid for (fleet nodes)
}
class RuntimeFull extends Error {}        // "no room just then": a wait passes it
class RuntimeUnsupported extends Error {} // what a runtime can't do: an optional operation,
                                          // or a world too big to move or pack
```

A.2 Edge protocol v1

Base `/edge/v1` on the control plane's internal listener, `Authorization: Bearer EDGE_TOKEN` (`packages/contracts/src/edge.ts`).

```
GET  /edge/v1/routes   → { routes: { hostname; destination: 'host:port';
                                     state?: 'asleep' | 'restarting' }[] }       (ETag, 304)
POST /edge/v1/wake     { hostname } → { ready: { destination } } | 'starting' | 'restarting'
                        | { denied: 'unknown' | 'suspended' | 'paused' | 'quota' | 'deleted' }
POST /edge/v1/idle     { hostname } → 204          // a hint; the idle policy decides
POST /edge/v1/sessions { edgeId, hostname, event: 'connect' | 'disconnect', player?, at } → 204
```

A.3 Node endpoint (fleet)

```
POST /fleet/v1/enroll
     { token, csrPem, facts: NodeFacts }
   → { nodeId, deploymentId, serverCertPem, clientCertPem, caPem, allowedClients,
       heartbeatSeconds }
POST /fleet/v1/nodes/:id/heartbeat
     { nodeId, sessionId, bootId, seq, daemonVersion, protocol, features, runtimeUp, reconciled,
       capacity, workloads: WorkloadReport[], issues, addresses? }
   → { lifecycle, fences: { workload, currentEpoch }[], heartbeatSeconds?, leaseSeconds?, renew? }
POST /fleet/v1/nodes/:id/renew
     { nodeId, csrPem } → { serverCertPem, clientCertPem, caPem }
```

B Database entities

52 tables in `packages/db/src/schema.ts`; 45 migrations (`0000_init` … `0044_backup_sha256`), applied by `packages/db/src/migrate.ts` as Fly's `release_command`, before any machine takes a new image. `0043` adds the key a spent enrollment token enrolled (`used_key_sha256`), `0044` an archive's sha256 to `backups`.

Table 26: Tables by group.

Group	Tables
Accounts and auth	`users`, `auth_sessions`, `auth_accounts`, `auth_verifications` (Better Auth), `platform_admins`, `account_standing` (plan, status, restrictions, overrides, extra units)
Servers	`minecraft_servers` (the aggregate: owner, slug, region, size, status, failure, revision, world; never a hostname), `retired_slugs`, `server_revisions` (immutable boot configs), `worlds`, `server_runtimes` (the binding: provider, opaque handle, applied spec, observed state; `move_to`)
Work	`server_operations` (kind, status, progress, owner's error, operator's detail; idempotency key per server); pg-boss's own schema
Access	`server_access`, `server_access_entries` (whitelist, ops, bans, pending states)
Worlds' copies	`backups` (snapshots and archives, trigger, status, handle or archive key), `stored_artifacts`, `pending_uploads`, `mod_uploads`, `pack_contents`, `pack_imports`
Catalog and curation	`catalog_projects`, `catalog_versions`, `catalog_refreshes`, `pack_checks`, `curated_releases`, `trusted_mod_projects`
Sharing and community	`public_listings`, `listing_reports`, `server_stars`, `server_notes`, `play_domain_joins`
Use and money	`power_intervals` (runs, with their `provider`), `server_usage_days`, `server_presence`, `server_activity` (both unlogged), `server_players`, `billing_subscriptions`, `billing_orders`
Platform	`platform_controls` (kill switches, caps), `platform_alerts`, `audit_log`, `realtime_certificate`
Runtimes	`runtime_rules` (placement rules), `runtime_decisions` (`placed`, `fell_back`, `move_requested`, `move_cancelled`, `moved`, `move_failed`)
Fleet	`fleet_nodes`, `fleet_node_tokens`, `fleet_placements`, `fleet_placement_history`, `fleet_observations`, `fleet_archives`, `fleet_endpoints`, `fleet_events`: read and written only by `infra/fleet`

Table 27: The fleet's tables. `packages/db/src/schema.ts:1168`.

Fleet table	Key columns
`fleet_nodes`	id, name, `region_key`, `api_address`, `edge_host`, `control_host`, `lifecycle` (active, draining, lost, retired), `lost_at`, `lost_reason`, `fenced_by`, certificate pins and expiries (current and next), `machine_id_sha256`, `boot_id`, session ids, `heartbeat_seq`, `last_heartbeat_at`, `runtime_up`, `reconciled`, `health`, capacity, features, issues, labels, `last_probe`, `daemon_version`. Never deleted.
`fleet_node_tokens`	`token_sha256` (unique), region, labels, `expires_at`, `used_at`, `used_by`, `used_key_sha256` (0043: the key it enrolled, which alone may ask again), `node_id` (re-enrollment), `created_by`.
`fleet_placements`	`workload` (key), `node_id`, `epoch`, `state` (placing, placed, released, displaced), region, memory, CPU, disk, ports, `handle`, `spec_digest`, `completing` (restore, move, recover), `restore_from`, `desired_power`, `power_changed_at` (0042), `move_requested_at`, `move_to`.
`fleet_placement_history`	(`workload`, `epoch`), node, reason (placed, restore, move, recover), started, ended, `end_reason` (moved, restored, released, lost, destroyed, refused).
`fleet_observations`	(`node_id`, `workload`), epoch, `superseded_by`, state, `spec_digest`, last report (with the node's issues for it), `observed_at`, `state_changed_at`. Written only when a report changes.
`fleet_archives`	id, workload, purpose (snapshot, move), epoch, node, `local_id`, `object_key`, `status` (creating, local, uploading, ready, failed, deleted), `sha256`, size, consistency (quiesced, stopped), captured and uploaded times, attempts, error.
`fleet_endpoints`	Which control-plane processes serve the node endpoint, seen every 5 s.
`fleet_events`	kind, node, workload, epoch, data; kept 90 days.

C Protocol operations

blocklyd's workload API (`blocklyd/src/api/mod.rs:460` full text: `blocklyd/docs/protocol.md`). Every mutating verb is idempotent and checks its epoch rule: `Exact` (the current copy only), `Teardown` (any copy, even superseded), `Place` (create or replace; a newer epoch takes over). A workload made with an epoch refuses a request without one (`428`).

Table 28: Operations.

Request	Rule	Notes
`PUT /v1/workloads/{id}`	Place	Ensure: create stopped, replace on a new spec or epoch (restart if it ran); `If-Match` / `If-None-Match`; disk floor, ports, image allowlist, memory admission when running.
`POST …/start`	Exact	Memory admission; `not_created`, `container_missing`, `port_conflict`.
`POST …/stop`, `…/kill`	Teardown	Graceful stop with the spec's signal and grace; kill is SIGKILL.
`DELETE /v1/workloads/{id}?data=keep|delete`	Teardown	`delete` needs the confirm header equal to the id; data goes to the trash.
`POST …/fence {currentEpoch}`	none	Mark superseded (in memory first), restarts off, stop; data kept.
`POST …/export {url, headers, quiesced, exclude, parts?}`	Exact	tar.gz to a presigned PUT; running only when quiesced; disk floor for the spool; over one PUT's limit (`transfer.max_put_mb`) in the `parts` offered, each part's ETag returned, or refused before sending without enough of them.
`POST …/restore {url, sha256?} | {snapshot}`	Exact	Stopped only; disk admitted; unpack beside, then one exchange.
`POST …/snapshots {id, quiesced}`	Exact	Local copy, FICLONE or copy; same id returns the existing one.
`GET …/snapshots`, `DELETE …/snapshots/{s}`, `POST …/snapshots/{s}/upload`	none	List; delete (`snapshot_busy` while uploading); upload as an export does, one at a time per snapshot (`snapshot_busy`).
`POST …/exec {command, timeoutSeconds}`	Exact	Running only; `Idempotency-Key`; timeout kills the process tree.
`GET …/logs`, `…/stats`, `/v1/workloads`, `/v1/node`, `/v1/health`	none	Reads.

Table 29: Error codes: the code is the contract. `blocklyd/src/api/error.rs:21`.

Code	Status	Meaning
`invalid_request`	422 / 400	Validation, with every field problem at once.
`stale_epoch`, `epoch_ahead`, `superseded`	409	Epoch refusals, with the epochs involved.
`epoch_required`	428	The workload has an epoch; the request named none.
`insufficient_capacity`, `no_free_ports`	409	No memory or no host port.
`insufficient_disk`	507	Below the disk floor, or a download that wouldn't fit above it.
`archive_too_large`	422	An archive over one PUT's limit with no parts, or too few, offered: refused before anything is sent, with its size.
`not_created`, `not_stopped`, `not_running`, `not_quiesced`, `container_missing`, `port_conflict`, `snapshot_busy`, `no_compute`, `name_taken`	409	State conflicts.
`precondition_failed`	412	`If-Match` mismatch.
`confirmation_required`	428	Data deletion without the confirm header.
`invalid_archive`, `checksum_mismatch`	422	A restore refused.
`runtime_unavailable`, `timeout`, `runtime_error`, `transfer_failed`, `internal`	503, 504, 502, 502, 500	Docker or transfer failures.
`forbidden`	403	A valid certificate for a name not allowed.

Features announced (instead of versions): `exec-idempotency-key`, `logs-follow-across-restarts`, `conditional-put`, `placement-epochs`, `data-transfer`, `local-snapshots`, `export-exclude`, `execution-lease`, `certificate-renewal`, `resume-after-reboot`, `multipart-upload`. The fleet places only on nodes with `placement-epochs`, `data-transfer`, `local-snapshots` and `export-exclude`, and offers parts only to nodes with `multipart-upload`.

D Configuration and environment

D.1 Control plane

Read once at boot, validated, and refused with every problem listed (`apps/control/src/config/schema.ts:109`).

Table 30: Control-plane settings. `apps/control/src/config/load.ts:110`.

Variable	Default	Meaning
`DEPLOYMENT_ID`	required	`[a-z0-9-]{2,16}`; names apps and resources, cookie prefix.
`ROLES`	`api,worker,realtime`	Which roles this process runs.
`DATABASE_URL`, `DATABASE_DIRECT_URL`	—, = URL	Pooled and direct (LISTEN, migrations).
`WEB_CANONICAL_ORIGIN`, `WEB_TRUSTED_ORIGINS`	—, empty	The web origin; extra trusted origins.
`API_LISTEN`, `INTERNAL_LISTEN`	`127.0.0.1:4000`, `127.0.0.1:4001`	Public API; internal listener (edge, operators).
`REALTIME_LISTEN`, `REALTIME_FALLBACK_LISTEN`, `REALTIME_PUBLIC_URL`, `REALTIME_FALLBACK_URL`, `REALTIME_TICKET_SECRET`, `REALTIME_TLS_MODE`	`:7443`, `:7444`, —, —, —, `pinned`	WebTransport and WebSocket; TLS pinned, provided or ACME.
`ACME_*`, `CLOUDFLARE_DNS_API_TOKEN`, `CLOUDFLARE_ZONE_ID`	—	Let's Encrypt via Cloudflare DNS-01, for realtime.
`PLAY_DOMAIN`, `PLAY_DOMAIN_ALIASES`, `PLAY_PORT`	—, empty, 25565	Where players address servers.
`REGIONS`	required	`key:Label` pairs; each maps on the default runtime.
`AUTH_SECRET`, `AUTH_GITHUB_*`, `AUTH_GOOGLE_*`, `AUTH_OAUTH_PROXY_SECRET`, `ADMIN_EMAILS`	—	Sign-in; OAuth providers optional; bootstrap admins.
`WEB_PROXY_SECRET`	unset	The web tier's word on a client's address.
`SMTP_URL`, `MAIL_FROM`, `MAIL_ACCOUNT_FROM`	—, —, `MAIL_FROM`	Email; sign-in's own from its own sender.
`EDGE_TOKEN`	—	Shared with the edge.
`RUNTIME_SECRETS_KEY`, `RUNTIME_SECRETS_PREVIOUS_KEYS`	—, empty	Versioned key for per-server secrets; rotation.
`RUNTIME_PROVIDER`, `RUNTIME_PROVIDERS`	`docker`, = default	Default runtime; every runtime run.
`RUNTIME_REGION_MAP`, `<NAME>_REGION_MAP`	—	Product region → provider region. The shared map is the default runtime's alone; every other runtime needs its own, and places only the regions it names.
`OPERATOR_TOKEN`	unset (off)	Operator APIs; required with the fleet.
`ARTIFACTS_RUNTIME_FACING_URL`, `ARTIFACTS_MIRROR`	—, false	Where game runtimes fetch jars; mirror upstream jars.
`ARCHIVE_S3_ENDPOINT`, `_RUNTIME_ENDPOINT`, `_BUCKET`, `_REGION`, `_ACCESS_KEY_ID`, `_SECRET_ACCESS_KEY`	unset (no archives), —, —, `auto`	The archive store.
`POLAR_ACCESS_TOKEN`, `POLAR_WEBHOOK_SECRET`, `POLAR_SERVER`, `POLAR_PRODUCTS`	unset (no billing)	`POLAR_SERVER` must be named: `sandbox` or `production`.
`CATALOG_USER_AGENT`	set	Names Blockly to the mod catalogs.

Table 31: Runtime settings. Not configurable: the edge's 25 s wake wait, boot timeouts (120 s running, 10 min ready), the hourly wake limit, 10 min host-loss grace.

Variable	Default	Runtime
`FLY_ORG`, `FLY_API_TOKEN`, `FLY_NATS_URL`, `FLY_MACHINE_LIMIT`, `FLY_PLATFORM_MACHINES`	—, —, `nats://[fdaa::3]:4223`, 50, 4	Fly (limit − platform ≥ 2).
`BOAT_API_TOKEN`, `BOAT_API_URL`, `BOAT_RUN_TTL_SECONDS`, `BOAT_START_RESERVE`	—, `https://boat.dev/api/v1`, none, 0.1	Boat.
`DOCKER_SOCKET`, `DOCKER_GAME_NETWORK`	`/var/run/docker.sock`, `blockly-games`	Docker (one region).
`FLEET_CA_CERT`, `FLEET_CA_KEY`, `FLEET_NODE_LISTEN`, `FLEET_ENDPOINT_HOSTS`	—, —, `[::]:8443`, —	Fleet trust and endpoint, IPv6 and IPv4.
`FLEET_PLACEMENT`, `FLEET_HEADROOM_MB`, `FLEET_MEMORY_OVERCOMMIT`, `FLEET_CPU_OVERCOMMIT`, `FLEET_CPU_PRESSURE`, `FLEET_DISK_OVERCOMMIT`, `FLEET_CPU_MILLIS_PER_GB`	`balanced`, 0, 4, 1, 0.8, 1, 250	Fleet placement.
`FLEET_HEARTBEAT_SECONDS`, `FLEET_LEASE_SECONDS`, `FLEET_SUSPECT_SECONDS`, `FLEET_UNAVAILABLE_SECONDS`, `FLEET_NODE_CERT_DAYS`, `FLEET_RENEW_DAYS`	5, 120, 15, 45, 30, 10	Fleet timing and certificates.

D.2 Edge agent

`CONTROL_URL`, `EDGE_TOKEN` (required); `EDGE_ID` (hostname), `ROUTES_POLL_MS` (1000), `IDLE_HINT_AFTER` (`10m`), `CONNECTION_RATE_LIMIT` (1000), `MINECRAFT_PORT` (25565), `ROUTER_PORT` (25561), `PROXY_PROTOCOL`, the per-address and per-server limits (`docs/configuration.md`), `AGENT_PORT` (8090), `NOTICE_PORT` (25564), `SLEEPING_PORT` (25563), `ROUTES_FILE`, `ROUTER_BIN`, and the four message texts (`ASLEEP_MOTD`, `LOADING_MOTD`, `RESTARTING_MOTD`, `RESTARTING_JOIN`) (`apps/edge/agent.ts:19`).

D.3 blocklyd

TOML, every table refusing unknown keys (`blocklyd/src/config.rs`).

Table 32: blocklyd's configuration.

Key	Default	Meaning
`deployment_id`, `state_dir`	required, `/var/lib/blocklyd`	Containers of other deployments are never touched.
`api.listen`, `api.max_body_bytes`, `api.shutdown_grace_seconds`	required, 2 MiB, 30	The mTLS API: a private address, never public.
`ops.listen`	`127.0.0.1:7070`	`/healthz`, `/readyz`, `/metrics`; loopback or a private address only, refused otherwise, because it names workloads.
`docker.socket`, `docker.network`, `docker.pull_timeout_seconds`	default socket, `blockly-workloads`, 1200	The bridge network is made with inter-container traffic off.
`network.port_range`, `edge_ips`, `control_ips`, `port_quarantine_seconds`	42000–42999, loopback, loopback, 600	Never 0.0.0.0 (since `800eca9`).
`workloads.allowed_images`	`itzg/minecraft-server:` (and `docker.io/…`)	Prefixes; everything else refused.
`workloads.user`, `data_owner`	`1000:1000`, = user	Never uid or gid 0, refused at start; data owner for userns remapping.
`workloads.read_only_rootfs`, `tmp_size_mb`, `default_pids_limit`, `log_max_size_mb`, `log_max_files`, `oom_score_adj`, `trash_retention_hours`, `min_memory_mb`	true, 256, 4096, 20, 5, 500, 24, 256	Host policy a request can't relax.
`capacity.reserved_memory_mb`, `allocatable_memory_mb`, `min_free_disk_mb`, `memory_overcommit`, `reserved_cpu_millis`	2048, none, 5120, 1.0, 1000	What the host keeps; running memory never overcommitted by default.
`intervals.resync_seconds`, `stats_seconds`, `disk_usage_seconds`	30, 15, 300	Reconcile and sampling.
`transfer.max_put_mb`	5115	The most one PUT carries, in MiB (R2's 5 GiB less 5 MiB); a larger archive goes in the parts a request offers.
`fleet.url`, `ca`, `enrollment_token_file`, `heartbeat_seconds`, `api_address`, `labels`, `restart_requires_contact_seconds`	required, required, none, 5, = `api.listen`, {}, 120	Fleet mode; without `[fleet]` blocklyd is a single-node daemon.

E Metrics and events

Table 33: Node metrics. `blocklyd/src/metrics.rs:142`.

blocklyd metric (`/metrics`)	Labels or meaning
`blocklyd_operations_total`, `blocklyd_operation_duration_seconds`	`op`, `outcome` (ok, refused, full, runtime_unavailable, error)
`blocklyd_http_requests_total`, `blocklyd_auth_failures_total`	method, route, status; reason
`blocklyd_reconciles_total`, `blocklyd_reconcile_duration_seconds`, `blocklyd_runtime_events_total`	outcome; —; action
`blocklyd_info`, `blocklyd_uptime_seconds`, `blocklyd_docker_up`	version, protocol, node, deployment
`blocklyd_workloads`, `blocklyd_ports_allocated`, `blocklyd_ports_capacity`, `blocklyd_issues`	by state (all 11)
`blocklyd_workload_memory_working_set_bytes`, `…_cpu_cores`, `…_data_bytes`	per workload
`blocklyd_process_resident_memory_bytes`, `blocklyd_process_cpu_seconds`	the daemon itself
`blocklyd_heartbeats_total`, `blocklyd_fleet_contact_age_seconds`, `blocklyd_fleet_lease_remaining_seconds`, `blocklyd_certificate_renewals_total`, `blocklyd_snapshot_bytes`	fleet mode (−1 before first contact or without a lease)

Table 34: Fleet events: for operators, never read back to decide anything. `apps/control/src/infra/fleet/sql.ts:69`.

Area	`fleet_events` kinds (47)
Nodes	`token.created`, `node.enrolled`, `node.enrollment_repeated`, `node.reenrolled`, `node.machine_id_reused`, `node.duplicate_identity`, `node.certificate_issued`, `node.certificate_renewed`, `node.addresses_changed`, `node.daemon_restarted`, `node.rebooted`, `node.health`, `node.draining`, `node.undrained`, `node.lost`, `node.returned_while_lost`, `node.reinstated`, `node.retired`, `node.quarantine_cleared`, `node.labels_changed`
Placements	`placement.created`, `placement.refused`, `placement.placed`, `placement.adopted`, `placement.rehomed`, `placement.restored`, `placement.released`, `placement.destroyed`, `placement.displaced`, `placement.move_requested`, `placement.move_declined`, `placement.moved_for_room`, `placement.finished_from_report`, `placement.abandoned`
Copies and epochs	`epoch.adopted`, `epoch.conflict`, `copy.orphaned`, `copy.fenced`, `copy.tidied`, `fork.detected`
Worlds	`snapshot.local`, `snapshot.uploaded`, `snapshot.upload_failed`, `snapshot.local_trimmed`, `move.exported`, `move.export_superseded`, `recover.older_copy`

- **Domain events** (NOTIFY `blockly_events`, hints of at most 8 kB): `server_changed`, `operation_progress`, `access_changed`, `backup_changed`, `listing_changed`, `presence`, `session_cap`.
- **Audit actions** include `server.runtime_moved`, `server.rebuilt_from_backup`, `server.wake_failed`, `server.stray_compute_stopped`, `server.relocation_scheduled`, `platform.controls_changed`, `operation.retried`, `operation.discarded`, `console.command`, `account.plan_changed`, `billing.checkout_started`, and the subscription changes in `app/billing/audit.ts` (`billing.subscribed`, `billing.cancel_scheduled`, `billing.subscription_lapsed`, `billing.subscription_ended` and the rest).
- **Admin alerts**: `blocked_operations`, `purge_overdue`, `catalog_stale`, `worlds_outgrow_plan`, `extra_play_unsent`, `extra_play_waiting`, `billing_order_unknown`.

F Benchmarks and measurements

Every number below came from one machine or a test; none from production. RESEARCH ONLY

Table 35: Measurements.

Where	Measured	Value
Test VM, 4 vCPU, ext4 virtual disk	Local snapshot of a 1 GB world (full copy, no reflink)	2.9 s, 378 MB/s
same	Archive (gzip −1, one thread); move archive without libraries	14.8 s (73 MB/s); 14.3 s
same	Restore from archive; from a node snapshot	10.8 s (100 MB/s); 6.1 s
same	Heartbeat of a node with 50 servers	10 ms mean, 20 ms p99
same	blocklyd binary; resident memory	8.3 MB; 7.5–9.5 MiB
Two-node experiment (Docker-in-Docker on one VM)	Enrollment, token to healthy	0.9–1.1 s
same	Move of a 116 MB / 653 MB archive	11.9 s / 52.6 s
same	Restore in place (116 MB); rebuild after loss	1.9 s; 1.2 s plus boot
Single-node VM, Minecraft 1.21.8, 3 GB, 2 CPUs	First boot to answering; restart to ready	29.4–37.1 s; 21.3–22.5 s
same	blocklyd restart to ready (one workload)	40–45 ms
Boat	Create to running; join on a sleeping server to running	34–40 s; 34 s
same	Backup; restore in the sandbox	4.0 s; 27.7–31.4 s
Fly staging	Volume snapshot point in time	Not point-in-time: writes 4.1 s after the ask were held; listed 66–70 s later
Fly ams	5 players spread, perf-1x 4 GB vs perf-2x	31.4 ms vs 33.8 ms per tick: a second core doesn't help

Not measured: reflink snapshots on a real XFS or btrfs disk, ZFS, density on a candidate dedicated CPU, wake times on Fly, running admission under real use, bandwidth.

G Test coverage

Table 36: Tests by area (TypeScript counts by grep: about 1,058 cases in 129 files). The full TypeScript suite when this was written: 1,043 passed, 28 skipped, 0 failed.

Area	Files	≈ Tests	Needs
`apps/control/src/app`	47	350	Postgres for 36 files; S3 for archives and moves
`apps/control/src/infra`	39	349	Fleet 99 (10 files), Fly 64, Boat 47; some need Docker, NATS, Pebble, S3
`apps/control/src/domain`, `minecraft`, `config`, `interfaces`	29	305	Pure
`apps/web`, `apps/edge`	14	54	—
blocklyd (Rust)	9 + unit	142	9 ignored: 8 need Docker and root (all 8 pass with them), 1 benchmark

Table 37: End-to-end evidence.

Suite	Proves	In CI
`fleet-runtime.e2e.test.ts` (11)	The production adapter against a real blocklyd: mTLS enrollment, placement, snapshot and restore through a real S3, a world over the node's one-PUT limit uploaded and exported in parts, adoption, SIGKILL of the daemon leaves the server running, re-enrollment, a lost node fenced, the operator API.	yes (`blocklyd` workflow)

Suite	Proves	In CI
`fleet-two-node.e2e.test.ts` (7)	Two nodes (Docker-in-Docker): operator move, drain, move for room, rebuild after loss, the lost node's copy fenced and the fork reported. "Tests the fleet's logic, not hardware failure."	no: needs root and dind
`hybrid.test.ts` (15)	Rules, allowlist, fallback, canary off, moves both ways through real S3, a failed move, overflow and its lack of fallback, a resting world holding no machine.	yes
blocklyd `tests/*`	Lifecycle, epochs, resume, reconcile, transfer, mTLS; Docker hardening as seen from inside a container.	yes (unprivileged, then Docker tests as root)
Manual	`smoke`, `staging-check`, `staging-waits`, `snapshot-probe`, the capacity harness.	no

CI: `ci.yml` (lint, typecheck, boundaries, deprecated APIs, Terraform validate, OpenAPI drift, the whole `bun test` against Postgres, NATS, S3 and Pebble); `blocklyd.yml` (`check`, `supply-chain`, `fleet-e2e`, `release`). Never tested: independent hosts, real reboots and power fencing, reflink on a real disk, ZFS, cgroup v1 vs v2 differences beyond CI's runner, IPv6 publishing, user namespaces.

H Glossary

Table 38: Words this document uses.

Term	Meaning here
Admission (running)	Only servers that run, start or are being made hold a node's memory and CPU; a sleeping one holds disk.
Archive	A gzip tarball of a world in the archive store; the format every runtime reads.
Archive store	The S3-compatible bucket (R2, Tigris, RustFS): backups' durable copies, resting worlds, moves.
Binding	`server_runtimes`: which runtime a server is on, and its handle.
Canary	A placement rule (or explicit moves) sending some new servers to a non-default runtime.
Copy	One node's container and data for a server at one epoch.
Displaced	A placement whose node an operator declared lost; it will be rebuilt.
Drain	A node takes no new placements; its servers move as they can.
Edge	mc-router and its agent: the only public Minecraft address.
Epoch	A per-server number that rises with every new home and is never reused.
Execution lease	How long after a heartbeat a node may restart or resume on its own (120 s).
Fence	"This server is at epoch N": an older copy is stopped and never runs again. Logical (by message) or physical (by the operator).
Fork	A lost node's copy reported when the node returns; kept for the owner to judge.
Handle	An opaque string a runtime issues for a server's compute.
Headroom	Memory kept free on every node for restores and moves (`FLEET_HEADROOM_MB`, 0).
Health	Derived from heartbeat age: healthy, degraded, suspect (15 s), unavailable (45 s). Never "lost".
Included hours	Units of play a plan includes (Free 20, Plus 60); 8 GB servers use two an hour.
Lifecycle (node)	active, draining, lost, retired: an operator's statement.
Lost	An operator's statement that a host is stopped and stays stopped.
Moved for room	A waking server moved to a node with room before it starts.
`NOWHERE`	`0.0.0.0:0`, the route of a sleeping server whose address isn't stable.
Operation	A unit of infrastructure work for one server, queued in order and retried.
Overcommit	`FLEET_MEMORY_OVERCOMMIT` (4): placed memory per node; blocklyd's `memory_overcommit` (1.0): running memory.
Park	Boat: wake a sleeping sandbox only for work, with a short time to live.
Placement	Fleet: choosing a node. Runtimes: choosing a runtime for a new server.
Quarantine	A node with two hosts beating as one; or a released port resting 600 s.
Reconstructible	Files the image remakes (jar, libraries), left out of moves.
Relocation	Moving a server: region, node, host loss, or runtime.
Rest (stored)	A world kept only in the archive store, with no compute or disk.
`RuntimeFull`	A runtime has no room just then; the server goes back to sleep.
Snapshot	A point-in-time copy of a world taken by its runtime.
Superseded	A copy a newer epoch replaced.
Upkeep	FleetRuntime's minute job: health, uploads, tidying.
Wake	A join on a sleeping server starts it, policy-checked, waited for up to 25 s.